

Day 36 — Sat 10 Oct

► EXAM DAY — CS Degree Paper-I

Countdown: 0 days to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

◎ Degree — Paper-I revision (data structures)

Source: `_Shared/CORE_08_DataStructures.md`

CORE 08 — Data Structures

Shared-core file 8 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points), **10/15-mark answer** (worked explanation + diagram).

0. Why this matters — where data structures appear in your three papers

Elective	Where	How heavy
CS Degree	P-I §2 — "Primitive data types, array, stack, queue, link list, trees, sorting and searching techniques, symbol tables, hashing etc. Analysing Algorithms, the big-Oh notation, divide and conquer & greedy method, Dynamic programming, Search and traversal techniques, NP completeness." A full dedicated unit , and it feeds straight into the Algorithms and Compiler units too.	Very heavy
Computer Forensic	TP-II §6 — "Data, Information, Definition of data structure. Arrays, stacks, queues, linked lists, trees, graphs, priority queues and heaps. File structures: Fields, records and files. Sequential, direct, index-sequential and relative files. Hashing, inverted lists and multi-lists. B-trees and B+ trees." Names B-trees, B+ trees, inverted lists, multi-lists explicitly.	Heavy
CS Diploma	Not a named unit — but C programming (P-II §5) leans on arrays, strings and structures, and the DBMS units assume file organisation.	Light (indirect)

Two of your three electives name data structures directly, and the topics overlap almost perfectly. The Forensic syllabus is the more demanding on *file structures* (inverted lists, multi-lists, B+ trees); the Degree syllabus is the more demanding on *algorithm analysis* (big-O, divide-and-conquer, greedy, DP). Study the union once. Worked numericals — an address calculation, a postfix evaluation, an AVL rotation, a Dijkstra table, a sort trace — are the highest-scoring descriptive answers because they are unambiguous to mark.

Topic checklist

- ☐ Data vs information; ADT vs data structure; linear vs non-linear; static vs dynamic
- ☐ **Arrays:** 1D/2D, **address calculation (row- vs column-major), worked**
- ☐ Sparse matrices — triplet and linked representations
- ☐ **Stacks:** operations, array & linked implementation, applications

- [] **Infix → postfix/prefix conversion, and postfix/prefix evaluation — worked**
- [] Recursion; recursion vs iteration; the call stack; tail recursion
- [] **Queues**: simple, **circular (with the full/empty problem)**, priority, deque
- [] **Linked lists**: singly, doubly, circular; header nodes; array-vs-list comparison
- [] **Tree terminology; binary tree properties and counting formulas**
- [] **Traversals (in/pre/post/level) — worked traces**
- [] **Reconstruction of a tree from two traversals — worked**
- [] **BST** — insert, delete (three cases), search
- [] **AVL trees — all four rotations (LL, RR, LR, RL) worked**
- [] **B-trees and B+ trees** — order, insertion with splitting, the difference
- [] **Heaps and heap sort** — build-heap, heapify, worked
- [] **Graphs**: adjacency matrix vs list; **BFS/DFS worked traces**
- [] Spanning trees; **Prim and Kruskal — worked**; **Dijkstra — worked table**; topological sort
- [] Searching: linear, binary (iterative + recursive, worked)
- [] **Every sort** with a trace + the **master complexity/stability table**
- [] **Hashing**: functions, load factor, chaining, linear/quadratic probing, double hashing — worked
- [] File structures: sequential, direct, index-sequential, relative, inverted, multi-list
- [] Asymptotic notation: **big-O**, **big-Ω**, **big-Θ**; best/average/worst

1. Fundamentals — data, ADT, and classification

Concept

Data are raw, unprocessed facts and figures (the number 42, the string "NPSC").

Information is data that has been processed, organised and given context so it becomes meaningful (42 marks out of 50 = 84%, a distinction). A **data structure** is a systematic way of **organising and storing data in memory so that it can be used efficiently** — the choice of structure directly determines how fast the operations you care about will run.

An **Abstract Data Type (ADT)** is the *specification* — **what** operations are available and what they do — deliberately kept separate from the *implementation* — **how** they are coded.

A "Stack ADT" promises `push`, `pop`, `peek`, `isEmpty` with LIFO behaviour; whether you build it on an array or a linked list is an implementation choice invisible to the user. **ADT = the interface; data structure = the realisation.** Saying that sentence earns the mark.

Classification

Axis	Categories
Primitive vs non-primitive	Primitive = directly operated on by the machine: <code>int</code> , <code>float</code> , <code>char</code> , <code>pointer</code> . Non-primitive = built from primitives: arrays, lists, trees, graphs
Linear vs non-linear	Linear — elements form a sequence, each has one predecessor and one successor: array, stack, queue, linked list. Non-linear — an element may have many neighbours: tree, graph
Static vs dynamic	Static — size fixed at compile time, contiguous memory: array. Dynamic — grows/shrinks at run time, scattered memory linked by pointers: linked list, tree, graph
Homogeneous vs heterogeneous	Homogeneous — all elements same type (array). Heterogeneous — mixed types (structure/record)

Operations common to all structures: traversal, insertion, deletion, searching, sorting, merging, updating.

Likely exam questions

- [5] Differentiate between data and information. Define a data structure and an ADT.
- [5] Classify data structures with examples. Distinguish linear from non-linear.
- [5] What is an abstract data type? Explain with the stack as an example.

MCQ traps

Trap	Correct answer
"Stack, queue, array, linked list are?"	Linear data structures.
"Tree and graph are?"	Non-linear .
"An array is static or dynamic?"	Static (fixed size, contiguous).
"ADT specifies?"	What operations do, not how they are implemented.
"int, char, float are?"	Primitive data types.

2. Arrays

Concept

An **array** is a collection of elements of the **same type** stored in **contiguous memory locations**, accessed by an **index**. Contiguity is the whole point: because the elements are adjacent and equal-sized, the machine can compute the address of any element **directly by arithmetic** — no searching — giving **O(1) random access**. That is the array's superpower and the reason address calculation is examined so heavily.

The trade-off: size is **fixed** (static), and insertion/deletion in the middle costs **O(n)** because every following element must shift.

2.1 Address calculation — 1D array (WORKED)

For a 1D array **A** with **base address B**, element size **w** bytes, and **lower bound LB** (the first index):

$$\text{Address}(A[i]) = B + (i - LB) \times w$$

Worked: `int A[50]` starts at address **2000**, **w = 4** bytes, indices start at **0**.

Find the address of `A[12]`.

$$\text{Address}(A[12]) = 2000 + (12 - 0) \times 4 = 2000 + 48 = 2048$$

Worked (1-based): An array with **LB = 1**, base 1000, **w = 2**. Address of the 8th element:

$$\text{Address}(A[8]) = 1000 + (8 - 1) \times 2 = 1000 + 14 = 1014$$

The single most common mistake is forgetting to subtract the lower bound. Always write the **(i - LB)** term explicitly.

2.2 Address calculation — 2D array (WORKED)

A 2D array `A[m][n]` (m rows, n columns) is stored in **one dimension** in memory, in one of two orders. Let element `A[i][j]`, base **B**, size **w**, row lower bound **LB_r**, column lower bound **LB_c**.

Row-major order (C, C++, Python) — store **row by row**, left to right:

$$\text{Address}(A[i][j]) = B + [(i - LB_r) \times n + (j - LB_c)] \times w$$

(n = number of columns)

Column-major order (FORTRAN, MATLAB, R) — store **column by column**, top to bottom:

$$\text{Address}(A[i][j]) = B + [(j - \text{LBc}) \times m + (i - \text{LBr})] \times w$$

(m = number of rows)

Worked (both). Array $A[1..4][1..5]$ (so m = 4 rows, n = 5 cols), base $B = 1000$, w = 4 bytes, lower bounds 1. Find the address of $A[3][4]$.

Row-major:

$$\begin{aligned} &= 1000 + [(3 - 1) \times 5 + (4 - 1)] \times 4 \\ &= 1000 + [2 \times 5 + 3] \times 4 \\ &= 1000 + [13] \times 4 \\ &= 1000 + 52 = 1052 \end{aligned}$$

Column-major:

$$\begin{aligned} &= 1000 + [(4 - 1) \times 4 + (3 - 1)] \times 4 \\ &= 1000 + [3 \times 4 + 2] \times 4 \\ &= 1000 + [14] \times 4 \\ &= 1000 + 56 = 1056 \end{aligned}$$

The examiner's tell: in row-major the **column count n** appears in the formula; in column-major the **row count m** appears. Memorise it as "*multiply by the count of whatever you finished storing first.*" Row-major finishes rows first, so you multiply the row index by the number of columns.

2.3 Rules for reading array problems

- If indices "start at 0", $\text{LB} = 0$; if the problem says $A[1..n]$, $\text{LB} = 1$.
- "Word" often means 2 bytes, "int" 4 bytes — but **use whatever size the question gives**.
- For a 3-D array $A[p][q][r]$ in row-major:
 $B + [(i) \cdot (q \cdot r) + (j) \cdot (r) + k] \times w$ (with lower bounds subtracted).

2.4 Advantages and disadvantages

Advantages	Disadvantages
O(1) random access by index	Fixed size — must know maximum in advance
Simple, cache-friendly (contiguous)	Insertion/deletion O(n) — shifting
No pointer overhead per element	Wasted space if under-filled; overflow if over-filled
Basis for other structures (heap, hash table, matrix)	Contiguous block must be found in memory

Likely exam questions

- **[5]** Derive the address calculation formula for a 1D array and solve a numerical.
- **[5]** Differentiate row-major and column-major storage with an example.
- **[10]** For $A[1..6][1..8]$, base 500, each element 4 bytes, find the address of $A[4][6]$ in both row-major and column-major order, showing all steps, and explain the difference.

MCQ traps

Trap	Correct answer
"Time to access $A[i]$?"	O(1) — direct address arithmetic.

"C stores 2D arrays in?"	Row-major order.
"FORTRAN/MATLAB store in?"	Column-major.
"Insertion in the middle of an array?"	$O(n)$ — shifting required.
"Row-major formula multiplies row index by?"	Number of columns (n).

3. Sparse matrices

Concept

A **sparse matrix** is one in which **most elements are zero**. Storing an $n \times n$ matrix that is 95% zeros in a full 2D array wastes enormous space and time. A sparse representation stores **only the non-zero elements**, plus enough bookkeeping to locate them.

Rule of thumb: if non-zeros $< n^2/3$, a sparse representation saves space.

3.1 Triplet (3-column / coordinate) representation

Store each non-zero as a triple (`row, column, value`). The **first row** of the triplet table is a header giving (`number of rows, number of columns, number of non-zero elements`).

Worked. The 4×5 matrix

```
0 0 3 0 0
0 5 0 0 0
0 0 0 0 7
2 0 0 0 0
```

has 4 non-zeros. Triplet form (using 0-based indices):

Row	Col	Value
4	5	4 ← header: 4 rows, 5 cols, 4 non-zeros
0	2	3
1	1	5
2	4	7
3	0	2

Space used: $(4 + 1) \times 3 = 15$ integers, versus $4 \times 5 = 20$ for the full matrix — and the saving grows dramatically as the matrix gets larger and sparser.

3.2 Linked-list representation

Each non-zero becomes a node holding (`row, col, value, next`). Nodes can be chained in a single list, or organised as one linked list per row (a **multi-list**, see §17), which speeds row-wise operations. Linked representation is preferred when the matrix changes over time (insertions/deletions) because no shifting is needed.

Likely exam questions

- **[5]** What is a sparse matrix? Represent a given matrix in triplet form.
- **[10]** Explain triplet and linked-list representations of a sparse matrix with an example, and compare their space requirements against the full 2D array.

MCQ traps

Trap	Correct answer
"A sparse matrix has mostly?"	Zero elements.
"Header row of the triplet stores?"	#rows, #columns, #non-zero elements .
"Triplet uses how many columns?"	Three — row, column, value.

4. Stacks

Concept

A **stack** is a **LIFO** (Last-In-First-Out) linear list where all insertions and deletions happen at **one end only**, called the **top**. Think of a stack of plates: you add and remove from the top. The last element pushed is the first popped.

Operations (all $O(1)$):

Operation	Effect
<code>push(x)</code>	Add <code>x</code> on top; error if overflow (array full)
<code>pop()</code>	Remove and return the top; error if underflow (empty)
<code>peek()</code> / <code>top()</code>	Return the top without removing it
<code>isEmpty()</code> / <code>isFull()</code>	Test conditions

Array implementation: keep an index `top`, initialised to **-1** (empty). `push` increments `top` then stores; `pop` returns then decrements. **Overflow** when `top == SIZE-1`; **underflow** when `top == -1`.

Linked implementation: push/pop at the **head** of a singly linked list — no fixed size, never overflows (until memory exhausts).

4.1 Applications of stacks

1. **Expression conversion and evaluation** (infix $\leftrightarrow$ postfix/prefix) — §4.2, §4.3.
2. **Function-call management** — the **run-time call stack** stores return addresses, local variables and parameters; this is what makes **recursion** possible (§4.4).
3. **Recursion** implementation.
4. **Backtracking** — maze solving, N-queens, DFS.
5. **Undo/redo** in editors; browser **back** button.
6. **Balanced-parenthesis checking**.
7. **Reversing** a string or list.

4.2 Infix $\rightarrow$ postfix / prefix conversion (WORKED)

Why bother? Infix (`A + B`) needs parentheses and precedence rules to be unambiguous.

Postfix (Reverse Polish, `A B +`) and **prefix** (Polish, `+ A B`) need **neither parentheses nor precedence** — a machine can evaluate them in a single left-to-right (or right-to-left) scan with one stack. Compilers convert to postfix for exactly this reason.

Operator precedence (high $\rightarrow$ low): $\wedge$ (exponent, right-associative) $>$ $*$ / $>$ $+$ $-$.
Parentheses override everything.

Algorithm — infix to postfix (using an operator stack):

1. Scan left to right.
2. **Operand** → output it directly.
3. (→ push.
4.) → pop and output until (is popped (discard the pair).
5. **Operator** o_1 → while the stack top is an operator of **greater-or-equal** precedence (for left-associative ops), pop and output; then push o_1 . (For right-associative $\wedge$, pop only strictly greater.)
6. End of input → pop and output everything remaining.

Worked: convert $A + B * C - D / E$ to postfix.

Symbol	Stack	Output
A	(empty)	A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
-	+	A B C * (pop *, its precedence $\geq$ -) then...
-	-	A B C * + (pop + too, equal precedence)
D	-	A B C * + D
/	- /	A B C * + D
E	- /	A B C * + D E
end	(empty)	A B C * + D E / -

Postfix = $A B C * + D E / -$

Worked with parentheses and $\wedge$: convert $(A + B) \wedge C * D$.

Symbol	Stack	Output
(	(	
A	(	A
+	(+	A
B	(+	A B
)	(empty)	A B +
$\wedge$	$\wedge$	A B +
C	$\wedge$	A B + C
*	*	A B + C $\wedge$ ($\wedge$ has higher precedence, popped)
D	*	A B + C $\wedge$ D
end	(empty)	A B + C $\wedge$ D *

Postfix = $A B + C \wedge D *$

Infix → **prefix** (the reliable method):

1. **Reverse** the infix expression (swap (and)).
2. Convert to postfix (treating ^ associativity carefully).
3. **Reverse** the result.

Worked: $A + B * C \rightarrow$ prefix. Reverse: $C * B + A$. Postfix of that: $C B * A +$. Reverse: $+ A * B C$. **Prefix = + A * B C**. (Check: $+ A (* B C) = A + (B * C)$. Correct.)

4.3 Evaluation of postfix and prefix (WORKED)

Postfix evaluation — scan left to right, one operand stack:

- Operand $\rightarrow$ push.
- Operator $\rightarrow$ pop **two** operands (op2 first, then op1), compute op1 (operator) op2, push the result.
- At the end the stack holds the single answer.

Worked: evaluate $5\ 6\ 2\ +\ *\ 12\ 4\ /\ -$.

Symbol	Action	Stack (bottom→top)
5	push	5
6	push	5 6
2	push	5 6 2
+	$6 + 2 = 8$	5 8
*	$5 * 8 = 40$	40
12	push	40 12
4	push	40 12 4
/	$12 / 4 = 3$	40 3
-	$40 - 3 = 37$	37

Answer = 37.

Prefix evaluation — scan RIGHT to left:

- Operand $\rightarrow$ push.
- Operator $\rightarrow$ pop two (op1 first this time, then op2), compute op1 (operator) op2, push.

Worked: evaluate $- * + 4\ 3\ 2\ 5$ right-to-left.

Symbol (R→L)	Action	Stack
5	push	5
2	push	5 2
3	push	5 2 3
4	push	5 2 3 4
+	$4 + 3 = 7$	5 2 7
*	$7 * 2 = 14$	5 14
-	$14 - 5 = 9$	9

Answer = 9. (Check: $- ((+ 4\ 3)\ 2)\ 5 = ((4+3)2) - 5 = 14 - 5 = 9$. Correct.)

4.4 Recursion

Recursion is a function calling itself, directly or indirectly, on a **smaller sub-problem** until it reaches a **base case** that stops the descent. Every recursive definition needs two parts, and stating both earns the mark:

1. **Base case** — the terminating condition (no further recursion).
2. **Recursive case** — the function expressed in terms of a smaller input.

Each call pushes a new **activation record (stack frame)** — return address, parameters, local variables — onto the **call stack**; returning pops it. Runaway recursion (missing/never-reached base case) causes **stack overflow**.

Classic example — factorial:

```
int fact(int n){
    if (n <= 1) return 1;      /* base case */
    return n * fact(n - 1);    /* recursive case */
}
```

$\text{fact}(4) \rightarrow 4 * \text{fact}(3) \rightarrow 4 * 3 * \text{fact}(2) \rightarrow 4 * 3 * 2 * \text{fact}(1) \rightarrow 4 * 3 * 2 * 1 = 24.$

Recursion vs iteration:

	Recursion	Iteration
Mechanism	Function calls itself	Loop repeats a block
Termination	Base case	Loop condition becomes false
Memory	Extra — a stack frame per call ($O(\text{depth})$)	Constant (one set of variables)
Speed	Slower (call overhead)	Faster
Code	Often shorter, clearer for tree/divide-and-conquer	Better for simple counting loops
Risk	Stack overflow	Infinite loop (no crash)

Tail recursion — the recursive call is the *last* action; a compiler can then reuse the frame, achieving iteration-like $O(1)$ space.

Likely exam questions

- **[5]** Define a stack. List its operations and any four applications.
- **[5]** Explain overflow and underflow with the array implementation of a stack.
- **[10]** Convert $(A + B) * C - D ^ E ^ F$ to postfix, showing the stack at each step.
- **[10]** Evaluate the postfix expression $6\ 2\ 3\ +\ -\ 4\ 2\ *\ +$ step by step.
- **[10]** What is recursion? Explain the role of the stack in recursion using factorial, and compare recursion with iteration.
- **[15]** Explain the stack ADT and its applications. Give the algorithm for infix-to-postfix conversion and use it to convert one expression, then evaluate the resulting postfix.

MCQ traps

Trap	Correct answer
"A stack works on which principle?"	LIFO.
"Both ends used?"	No — one end (top) only.

"top initialised to?"	-1 (empty stack).
"Postfix of A+B*C?"	A B C * +.
"Which needs no parentheses?"	Postfix and prefix.
"Recursion uses which structure internally?"	Stack (the call stack).
"Deleting from an empty stack causes?"	Underflow.
"Postfix is also called?"	Reverse Polish Notation (RPN).

5. Queues

Concept

A **queue** is a **FIFO** (First-In-First-Out) linear list: insertions happen at the **rear** (enqueue) and deletions at the **front** (dequeue) — opposite ends. Like a queue at a ticket counter: first to arrive is first served.

Two indices: **front** (points at the element to be removed next) and **rear** (the last element inserted). Applications: **CPU scheduling** (ready queue), **printer/disk spooling**, **BFS traversal**, **buffering** (I/O, keyboard, streaming), request handling in servers.

5.1 Simple (linear) queue — and its flaw

Array of size `n`, `front = rear = -1` when empty. Enqueue increments `rear`; dequeue increments `front`. **The flaw**: after several dequeues, `front` marches forward and the vacated cells at the start **cannot be reused** — the queue reports "full" (`rear == n-1`) while the front cells sit empty. This wasted space is called **false overflow** (rightward drift). The **circular queue** fixes it.

5.2 Circular queue (WORKED)

Treat the array as a **ring**: after the last index, wrap to index 0 using **modulo n**.

- Enqueue: `rear = (rear + 1) % n; A[rear] = x;`
- Dequeue: `x = A[front]; front = (front + 1) % n;`

The full-vs-empty ambiguity — the classic exam point. If you only track `front` and `rear`, the condition `rear + 1 == front` (mod `n`) means *full*, but a genuinely empty queue can look the same. Two standard fixes:

1. **Leave one cell empty** — declare full when `(rear + 1) % n == front`; empty when `front == rear`. (Wastes one slot; simplest.)
2. **Keep a count** of elements — full when `count == n`, empty when `count == 0`.

Worked ($n = 5$, method 1, indices 0–4). Start empty: `front = rear = 0`.

Operation	front	rear	Contents
enqueue 10	0	1	[10]
enqueue 20	0	2	[10 20]
enqueue 30	0	3	[10 20 30]
enqueue 40	0	4	[10 20 30 40] — now $(4+1)\%5 == 0 == \text{front} \Rightarrow$ FULL (one slot left empty)

dequeue → 10	1	4	[20 30 40]
enqueue 50	1	0	[20 30 40 50] — rear wrapped to 0

5.3 Priority queue

Elements carry a **priority**; **dequeue removes the highest-priority element**, not the oldest.

Ties are broken FIFO. Used in **CPU scheduling (priority scheduling)**, **Dijkstra** and **Prim**, **Huffman coding**, event simulation.

Implementations: unsorted array ($O(1)$ insert, $O(n)$ delete-max), sorted array ($O(n)$ insert, $O(1)$ delete-max), or — the efficient one — a **binary heap** ($O(\log n)$ both; see §10).

5.4 Deque (double-ended queue)

Insertion and deletion allowed at **both** ends. Operations: `insertFront`, `insertRear`, `deleteFront`, `deleteRear`. Two restricted forms:

- **Input-restricted deque** — insertion at one end only, deletion at both.
- **Output-restricted deque** — deletion at one end only, insertion at both.

A deque can emulate **both a stack and a queue**, which is a favourite MCQ.

Comparison

Type	Insert	Delete	Order out
Simple queue	rear	front	FIFO
Circular queue	rear (wraps)	front (wraps)	FIFO, reuses space
Priority queue	by priority	highest priority first	by priority
Deque	both ends	both ends	flexible

Likely exam questions

- **[5]** Define a queue. Distinguish it from a stack.
- **[5]** What is the drawback of a linear queue? How does a circular queue overcome it?
- **[5]** Differentiate priority queue and deque.
- **[10]** Explain the circular queue with enqueue/dequeue algorithms and the full/empty conditions; trace a sequence of operations on a size-5 queue.
- **[10]** Explain the types of queues (simple, circular, priority, deque) with diagrams.

MCQ traps

Trap	Correct answer
"A queue follows?"	FIFO .
"Insertion and deletion in a queue happen at?"	Rear and front respectively (different ends).
"Circular queue full condition (one-slot method)?"	$(\text{rear} + 1) \% n == \text{front}$.
"Which can act as both stack and queue?"	Deque .
"Highest priority served first — which queue?"	Priority queue .
"BFS uses which structure?"	Queue . DFS uses a stack .

6. Linked lists

Concept

A **linked list** is a **dynamic** linear structure in which elements (**nodes**) are stored at scattered memory locations and threaded together by **pointers**. Each node holds **data** plus the **address of the next node**. The list is reached through an external pointer, **HEAD**; the last node's link is **NULL**, marking the end. Because nodes are allocated on demand, the list **grows and shrinks at run time** — no fixed size, no shifting.

The core trade-off vs arrays: a linked list gives **O(1) insertion/deletion** once you hold the relevant node (just re-point links, no shifting) but **loses O(1) random access** — to reach the i-th node you must **traverse** from HEAD, which is **O(n)**.

6.1 Singly linked list (SLL)

Each node: `[data | next]`. Links point **one way** only. You can move forward, never back.

```
HEAD → [10 | •] → [20 | •] → [30 | NULL]
```

Operations: insert at head O(1); insert at end O(n) (must walk to the tail unless a tail pointer is kept); delete a node O(1) if you have its predecessor; search O(n).

6.2 Doubly linked list (DLL)

Each node: `[prev | data | next]` — **two** links, so traversal is possible in **both directions**. Costs one extra pointer per node, but makes deletion easier (you have the predecessor for free) and enables backward scans.

```
NULL ← [• | 10 | •] ⇌ [• | 20 | •] ⇌ [• | 30 | •] → NULL
```

6.3 Circular linked list (CLL)

The **last node points back to the first** (in a circular DLL, first's `prev` points to last too). There is no NULL end; any node can be a starting point, and you can loop around forever — useful for **round-robin scheduling** and buffering. Care needed to avoid infinite loops (stop when you return to the start).

6.4 Header (dummy) node

An extra node at the front that holds no data (or holds the list length). It removes the special case of "inserting/deleting at the head", simplifying code.

6.5 Array vs linked list — the standard comparison table

Criterion	Array	Linked list
Memory	Contiguous , fixed at compile time	Scattered , allocated at run time
Size	Static — fixed	Dynamic — grows/shrinks
Access to i-th element	O(1) random access	O(n) — must traverse
Insertion/deletion (middle)	O(n) — shift elements	O(1) once the node is located (re-point links)
Memory per element	Just the data	Data + pointer(s) — extra overhead

Memory utilisation	Can waste (over-allocated) or overflow	Uses exactly what is needed
Cache locality	Excellent (contiguous)	Poor (scattered)
Ease of implementation	Simple	More complex (pointer handling)

One-line summary examiners like: "Use an array when you mostly index/read and know the size; use a linked list when you mostly insert/delete and the size is unpredictable."

6.6 Applications

Implementing stacks, queues and dequeues; **polynomial representation and arithmetic**; **sparse matrices**; dynamic memory management (free lists); **hash-table chaining**; adjacency lists for graphs; music playlists (CLL); undo (DLL).

Likely exam questions

- **[5]** Differentiate an array and a linked list (tabular).
- **[5]** Explain singly, doubly and circular linked lists with diagrams.
- **[5]** Write the algorithm to insert a node at the beginning of a singly linked list.
- **[10]** Explain the doubly linked list. Give algorithms to insert and delete a node, and state its advantages over a singly linked list.
- **[15]** Compare arrays and linked lists in detail. Explain the types of linked lists with diagrams and list four applications of linked lists.

MCQ traps

Trap	Correct answer
"Random access is possible in?"	Array ($O(1)$); not a linked list.
"Last node's next in an SLL?"	NULL .
"Which list allows backward traversal?"	Doubly linked list.
"Extra memory in a linked list is for?"	The pointer(s) .
"Round-robin scheduling suits which list?"	Circular linked list.
"Insertion at head of a list is?"	$O(1)$.

7. Trees — terminology and binary-tree properties

Concept

A **tree** is a **non-linear**, **hierarchical** structure: a set of nodes with one special node called the **root**, and the rest partitioned into disjoint subsets each of which is itself a tree (the subtrees). It has **no cycles** — a tree with n nodes has exactly **$n - 1$ edges**.

7.1 Terminology (memorise — pure MCQ fuel)

Term	Meaning
Root	The topmost node; has no parent

Parent / child	An immediate predecessor / successor
Siblings	Nodes with the same parent
Leaf (external/terminal)	A node with no children
Internal node	A node with at least one child
Degree of a node	Number of children it has
Degree of a tree	Maximum degree of any node
Level	Root is level 0 (some texts: level 1); each step down adds 1
Height / depth of tree	Number of edges on the longest root-to-leaf path (a single node has height 0)
Ancestor / descendant	Any node on the path up to the root / down from a node
Subtree	A node together with all its descendants
Forest	A set of disjoint trees (remove the root of a tree → a forest)

7.2 Types of binary tree

A **binary tree** is a tree in which every node has **at most two children** (left and right).

Type	Definition
Full (strict/proper)	Every node has 0 or 2 children (never 1)
Complete	All levels full except possibly the last , which is filled left to right . (This is the shape used for heaps and array storage.)
Perfect	All internal nodes have 2 children and all leaves are at the same level
Skewed	Every node has only one child — degenerates to a linked list (left-skewed or right-skewed)
Balanced	Height $\approx \log_2 n$; heights of left and right subtrees differ by a bounded amount (AVL: ≤ 1)

7.3 Binary-tree properties (the counting formulas — WORKED)

For a binary tree:

1. **Maximum nodes at level ℓ** (root = level 0) = 2^ℓ .
2. **Maximum nodes in a tree of height h** = $2^{(h+1)} - 1$.
3. **Minimum height** for n nodes = $\lceil \log_2(n+1) \rceil - 1$; a tree with n nodes has height at least $\lfloor \log_2 n \rfloor$.
4. In a **full/strict** binary tree, **number of leaf nodes = number of internal (degree-2) nodes + 1**, i.e. **$L = I + 1$** .
5. A binary tree with **n** nodes has **$n + 1$ NULL (external) links** (and $n - 1$ real edges).

Worked (property 4). A full binary tree has 8 internal (2-child) nodes. Leaves = $8 + 1 = 9$; total nodes = $8 + 9 = 17$.

Worked (property 2). A perfect binary tree of height 3 has $2^{(3+1)} - 1 = 15$ nodes, of which $2^3 = 8$ are leaves.

7.4 Array vs linked representation of a binary tree

- **Array (sequential):** store the root at index 1; for node at index i , **left child** = $2i$, **right child** = $2i + 1$, **parent** = $\lfloor i/2 \rfloor$. Efficient for **complete** trees (heaps); wasteful for skewed trees (many empty slots).
- **Linked:** each node `[left | data | right]`. Flexible for any shape; the usual choice.

Likely exam questions

- **[5]** Define: root, leaf, degree, height, sibling, forest.
- **[5]** Differentiate full, complete and perfect binary trees.
- **[5]** State and prove: a full binary tree with l internal nodes has $l + 1$ leaves.
- **[10]** Explain the array and linked representations of a binary tree with examples, and give the index relations for the array form.

MCQ traps

Trap	Correct answer
"Max nodes at level ℓ (root = 0)?"	2^ℓ .
"Max nodes in a tree of height h ?"	$2^{h+1} - 1$.
"A tree with n nodes has how many edges?"	$n - 1$.
"In the array form, left child of node i ?"	$2i$ (root at 1).
"Node with no children?"	Leaf .
"Full binary tree: every node has?"	0 or 2 children.

8. Binary tree traversals — with worked traces and reconstruction

Concept

Traversal = visiting every node exactly once in a systematic order. For binary trees there are three **depth-first** orders, defined by *when the root is visited relative to its subtrees*, plus one **breadth-first** order.

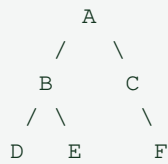
Traversal	Order	Rule
Inorder	Left, Root, Right	Visit left subtree, then root, then right subtree
Preorder	Root, Left, Right	Visit root first
Postorder	Left, Right, Root	Visit root last
Level-order (BFS)	Level by level, left to right	Uses a queue

The one fact that unlocks the topic: *the position of "Root" in the name tells you when to print the root.* **Inorder** → root in the **middle**; **preorder** → root **first**; **postorder** → root **last**. Left always precedes right.

Key result: the **inorder traversal of a BST yields the elements in sorted (ascending) order**. That is why inorder matters.

8.1 Worked traces

Consider this binary tree:



- **Inorder (L,Root,R)**: left of A first. Left subtree of A is rooted at B: (inorder of B) = D, B, E. Then A. Then right subtree (C): inorder of C = C, F.
→ **D B E A C F**
- **Preorder (Root,L,R)**: A, then preorder(B) = B D E, then preorder(C) = C F.
→ **A B D E C F**
- **Postorder (L,R,Root)**: postorder(B) = D E B, postorder(C) = F C, then A.
→ **D E B F C A**
- **Level-order**: A / B C / D E F → **A B C D E F** (uses a queue: dequeue a node, enqueue its children).

8.2 Reconstruction from two traversals (WORKED)

The rule: you can rebuild a unique binary tree from **(inorder + preorder)** or **(inorder + postorder)**. **Inorder is essential** — it tells you the left/right split. Preorder+postorder **cannot** reconstruct a general binary tree uniquely (only a full one).

Method (inorder + preorder):

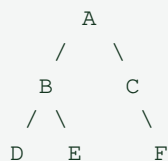
1. The **first** element of **preorder** is the **root**.
2. Find that root in **inorder**: everything to its **left** is the left subtree, everything to its **right** is the right subtree.
3. Recurse on each part (the next preorder elements fill the left subtree first).

Worked. Given:

```
Inorder  : D B E A C F
Preorder : A B D E C F
```

- Preorder[0] = **A** → root. In inorder, **D B E** | **A** | **C F** ⇒ left = {D,B,E}, right = {C,F}.
- Left subtree — preorder segment **B D E**, inorder **D B E**: root **B**; inorder **D** | **B** | **E** ⇒ left leaf **D**, right leaf **E**.
- Right subtree — preorder segment **C F**, inorder **C F**: root **C**; inorder **C** | **F** ⇒ nothing left, right child **F**.

Reconstructed tree:



— exactly the original. (Verify by taking its postorder: D E B F C A. ✓)

Method (inorder + postorder) is identical except the **root is the LAST** element of **postorder**, and you process the right subtree first when recursing.

Likely exam questions

- [5] Write the inorder, preorder and postorder traversals of a given tree.
- [5] State which pairs of traversals uniquely determine a binary tree, and why inorder is indispensable.
- [10] Given inorder **D B E A F C** and preorder **A B D E C F**, reconstruct the binary tree, showing each step, and give its postorder.
- [10] Explain the four traversal methods with an example tree, and state which traversal of a BST gives sorted output.

MCQ traps

Trap	Correct answer
"Inorder of a BST gives?"	Sorted (ascending) order.
"First node in preorder is the?"	Root.
"Last node in postorder is the?"	Root.
"Level-order traversal uses a?"	Queue.
"Which pair cannot uniquely rebuild a general tree?"	Preorder + postorder.
"Root visited last in?"	Postorder.

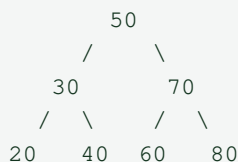
9. Binary search trees (BST) and AVL trees

9.1 BST

A **binary search tree** is a binary tree with the **ordering invariant**: for every node, **all keys in its left subtree are smaller, and all keys in its right subtree are larger** (no duplicates, conventionally). This makes search, insert and delete **$O(h)$** — $O(\log n)$ when the tree is balanced, but **$O(n)$ if it degenerates into a skewed tree** (e.g. inserting sorted data).

Search / insert: compare the key with the root; go left if smaller, right if larger; repeat. Insertion always adds a **new leaf** at the position where the search fails.

Worked insert. Insert **50, 30, 70, 20, 40, 60, 80** into an empty BST:

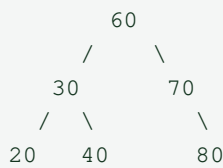


(50 is root; $30 < 50 \rightarrow$ left; $70 > 50 \rightarrow$ right; $20 < 30 \rightarrow$ left of 30; etc.)

Deletion — three cases (the examined part):

1. **Leaf node** — simply remove it.
2. **One child** — replace the node with its single child.
3. **Two children** — replace the node's key with its **inorder successor** (the smallest key in the right subtree) or **inorder predecessor** (largest in the left subtree), then delete that successor/predecessor node (which has at most one child).

Worked (case 3). Deleting **50** from the tree above: inorder successor = smallest in right subtree = **60**. Copy 60 into the root, then delete the old 60 (a leaf):



9.2 AVL trees — the four rotations (WORKED)

An **AVL tree** (Adelson-Velsky & Landis, **1962** — the first self-balancing BST) is a BST that additionally keeps, for **every** node, a **balance factor**

BF = height(left subtree) – height(right subtree) $\in \{-1, 0, +1\}$.

If an insertion or deletion makes any BF equal to **+2 or -2**, the tree is rebalanced by a **rotation**, restoring $O(\log n)$ height. There are exactly **four cases**, named by the direction of the imbalance:

Case	When it happens	Fix
LL (left-left)	Inserted into the left subtree of the left child (BF = +2, child BF = +1)	Single right rotation
RR (right-right)	Inserted into the right subtree of the right child (BF = -2, child BF = -1)	Single left rotation
LR (left-right)	Inserted into the right subtree of the left child (BF = +2, child BF = -1)	Left rotation on child, then right rotation (double)
RL (right-left)	Inserted into the left subtree of the right child (BF = -2, child BF = +1)	Right rotation on child, then left rotation (double)

Rule of thumb: the two single rotations (LL, RR) fix "straight-line" imbalances; the two doubles (LR, RL) fix "zig-zag" imbalances by first straightening the zig-zag into a straight line, then doing a single rotation.

Worked LL — insert 30, 20, 10:

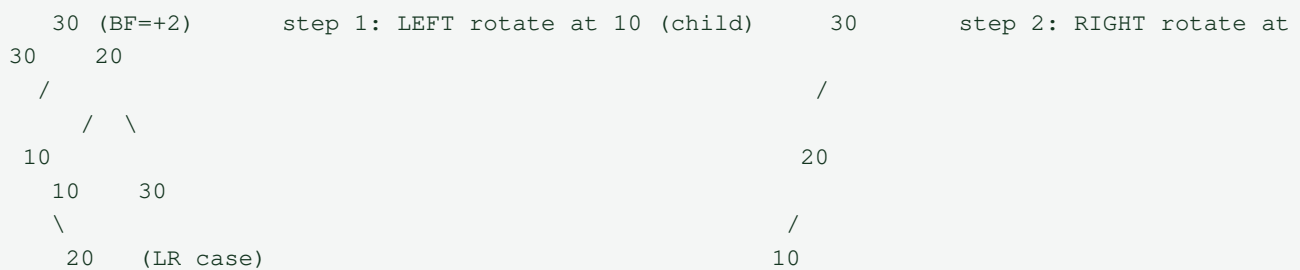
Insert 30, 20: 30 Insert 10: 30 (BF=+2) → single RIGHT rotation at 30
 → 20
 / /
 / \ /
 10 30 20
 /
 10 (LL case)

Result: 20 is root, 10 left, 30 right. Balanced.

Worked RR — insert 10, 20, 30:

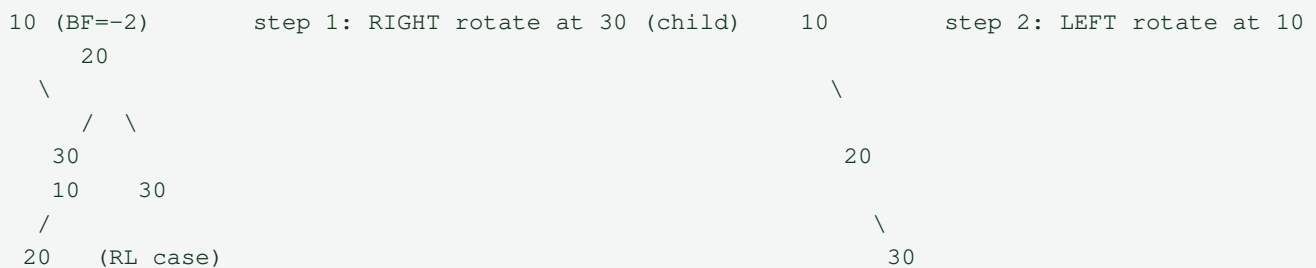
10 (BF=-2) single LEFT rotation at 10 20
 \ / \
 20 10 30
 \ /
 30 (RR case)

Worked LR — insert 30, 10, 20:



Result root = 20, children 10 and 30.

Worked RL — insert 10, 30, 20:



Result root = 20.

A larger worked insertion sequence (good for 15 marks): insert 10, 20, 30, 40, 50, 25.

- 10,20,30 → RR at 10 → root 20 (10,30).
- Insert 40 → right of 30, still balanced.
- Insert 50 → 30 becomes BF -2 (RR) → left rotate at 30 → subtree 40(30,50). Tree: 20 root, left 10, right 40(30,50).
- Insert 25 → goes left of 30. Now 40 has BF +2 with left-child 30 having BF +1... check the first unbalanced ancestor from the new node: node 40 has BF = height(left=30-subtree=2) – height(right=50=1) = +... trace carefully and apply the matching rotation. (The point the examiner wants: **identify the lowest unbalanced node, classify LL/RR/LR/RL, rotate.**)

Why AVL matters: it **guarantees** $O(\log n)$ search/insert/delete by keeping height $\leq \sim 1.44 \log_2 n$, whereas a plain BST can degrade to $O(n)$. The price is the rotation bookkeeping on every update.

Likely exam questions

- **[5]** Define a BST. Insert a given sequence and draw the tree.
- **[5]** Explain BST deletion with the three cases.
- **[5]** What is a balance factor? State the four AVL rotation cases.
- **[10]** Construct an AVL tree by inserting 21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7, showing each rotation. (Any given sequence — method is what is marked.)
- **[15]** Explain AVL trees. Describe all four rotations with diagrams, and build an AVL tree from a given insertion sequence, showing balance factors and rotations at each step.

MCQ traps

Trap	Correct answer
"Inorder of a BST is?"	Sorted.
"AVL balance factor range?"	-1, 0, +1.

"LL imbalance is fixed by?"	Single right rotation .
"LR needs?"	Two rotations (left then right).
"Worst-case search in a plain BST?"	O(n) (skewed). AVL guarantees O(log n) .
"Who invented AVL / in which year?"	Adelson-Velsky & Landis, 1962 .
"Deleting a two-child BST node uses?"	Inorder successor or predecessor .

10. Heaps and heap sort

Concept

A **binary heap** is a **complete binary tree** satisfying the **heap property**:

- **Max-heap**: every parent $\geq$ its children $\Rightarrow$ the **maximum is at the root**.
- **Min-heap**: every parent $\leq$ its children $\Rightarrow$ the **minimum is at the root**.

Because it is complete, a heap is stored in an **array** with no pointers: for index i (1-based), **left child = $2i$, right child = $2i+1$, parent = $\lfloor i/2 \rfloor$** . A heap is the standard efficient **priority queue** (§5.3): insert and delete-max/min are **O(log n)**; peek-max is **O(1)**.

10.1 Operations

- **Insert**: place the new key at the next array slot (bottom), then **sift up (bubble up)** — swap with the parent while it violates the heap property. **O(log n)**.
- **Delete-max (extract root)**: swap root with the last element, remove the last, then **sift down (heapify)** the new root — swap with its larger child while it violates the property. **O(log n)**.
- **Build-heap**: heapify the array bottom-up from index $\lfloor n/2 \rfloor$ down to 1. Cost is **O(n)** (tighter than the naive $O(n \log n)$).

10.2 Heap sort (WORKED)

Idea: (1) build a **max-heap** from the array; (2) repeatedly **swap the root (largest) with the last element**, shrink the heap by one, and **heapify** the root. After $n-1$ rounds the array is sorted ascending.

Worked. Sort 4, 10, 3, 5, 1.

Build max-heap (from array [4,10,3,5,1], heapify from index 2 down):

- Heapify index 2 (value 10, children 5,1): 10 already $\geq$ both $\rightarrow$ no change.
- Heapify index 1 (value 4, children 10,3): largest child 10 $>$ 4 $\rightarrow$ swap $\rightarrow$ [10,4,3,5,1]; now index 2 (4) has children 5,1: 5 $>$ 4 $\rightarrow$ swap $\rightarrow$ [10,5,3,4,1].
- Max-heap: **[10, 5, 3, 4, 1]**.

Sort phase:

| Step | Action | Array |

---|---|---

| 1 | swap root 10 with last (1); heapify [1,5,3,4] | [5,4,3,1 | 10] |

| 2 | swap 5 with 1; heapify [1,4,3] | [4,1,3 | 5 10] |

| 3 | swap 4 with 3; heapify [3,1] | [3,1 | 4 5 10] |

| 4 | swap 3 with 1 | [1 | 3 4 5 10] |

Sorted: 1 3 4 5 10.

Complexity: build $O(n)$ + n heapify $O(\log n)$ = **$O(n \log n)$** in **all** cases (best, average, worst) — its selling point. **In-place** ($O(1)$ extra), but **not stable**.

Likely exam questions

- [5] Define a max-heap and a min-heap. Give the array index relations.
- [5] Insert 15 into the max-heap [40,30,20,10,25]; show sift-up.
- [10] Explain heap sort with an algorithm; sort 12, 11, 13, 5, 6, 7 showing the heap at each step.
- [10] How is a priority queue implemented using a heap? Give the complexities.

MCQ traps

Trap	Correct answer
"Root of a max-heap is the?"	Maximum.
"A heap is which kind of binary tree?"	Complete.
"Heap sort time complexity (worst)?"	$O(n \log n)$.
"Is heap sort stable?"	No. In-place: yes.
"Build-heap cost?"	$O(n)$.
"Parent of index i (1-based)?"	$\lfloor i/2 \rfloor$.

11. B-trees and B+ trees

Concept — why they exist

BSTs and AVL trees assume the data sit in **RAM**. Databases and file systems store indexes on **disk**, where a single seek is $\sim 100,000\times$ slower than a memory access. The cost is dominated by the **number of disk blocks read**, i.e. the **height** of the tree. So we want a **short, bushy, balanced** tree with a **high branching factor (order)** — one node = one disk block holding *many* keys. That is the **B-tree**.

11.1 B-tree

A **B-tree of order m** is a balanced multi-way search tree in which:

1. Every node holds **at most $m - 1$ keys** and **at most m children**.
2. Every node (except root) holds **at least $\lceil m/2 \rceil - 1$ keys** (so $\geq \lceil m/2 \rceil$ children) — the **half-full** rule that keeps it balanced.
3. Keys within a node are **sorted**; children hang between/around the keys like a multi-way BST.
4. **All leaves appear at the same level** — the tree is **perfectly height-balanced**.
5. A non-leaf node with k keys has exactly **$k + 1$ children**.

Insertion: always insert into the correct **leaf** (found by search). If the leaf overflows (m keys), **split** it at the **median**: the median key moves **up** into the parent, and the node divides into two. Splits can cascade upward; if the root splits, the tree **grows in height by one** — this is the only way a B-tree height increases, and it grows from the **root**, which is why it stays balanced.

Worked (order 3 = 2-3 tree, max 2 keys/node). Insert 10, 20, 5, 6, 12, 30, 7, 17:

- 10, 20 → node [10 20].

- Insert 5 → [5 10 20] overflows (3 keys); median **10** goes up:

[10] / \ [5] [20]

- Insert 6 → leaf [5 6]. Insert 12 → leaf [12 20]. Insert 30 → [12 20 30] overflows; median **20** goes up to root [10 20]:

[10 20] / | \ [5 6] [12] [30]

- Insert 7 → leaf [5 6 7] overflows; median **6** up → root [6 10 20]... and so on. **The marked skill: insert into a leaf, split on overflow, push the median up.**

11.2 B+ tree

A **B+ tree** is the variant used by virtually all real databases (and NTFS, etc.). Two differences from a B-tree, both worth stating:

1. **All actual data/record pointers live only in the LEAF nodes.** Internal nodes store **only keys as an index/routing map** — so a key may appear twice (once as a router, once in a leaf). This lets each internal block index **more** children (no data taking up room), giving a shorter tree.
2. **The leaves are linked together in a sorted linked list.** This makes **range queries and sequential/ordered scans extremely efficient** — find the start leaf, then walk the link chain — which a plain B-tree cannot do without repeated root-to-leaf traversals.

11.3 B-tree vs B+ tree

Criterion	B-tree	B+ tree
Data stored in	All nodes (internal + leaf)	Leaf nodes only
Internal nodes	Hold keys and data	Hold keys only (index)
Key duplication	No	Yes (routing keys repeat in leaves)
Leaves linked?	No	Yes — sorted linked list
Range/sequential query	Poorer	Excellent
Tree height	Slightly taller	Shorter (more keys per internal node)
Search cost	Can stop early (data in internal node)	Always goes to a leaf (uniform)
Used by	Some file systems	Most DBMS indexes, NTFS

Likely exam questions

- [5] What is a B-tree of order m? State its properties.
- [5] Differentiate B-tree and B+ tree.
- [5] Why are B-trees preferred for database indexing over AVL trees?
- [10] Construct a B-tree of order 3 by inserting 1, 2, 3, 4, 5, 6, 7, showing splits.
- [10] Explain the B+ tree structure and how it supports efficient range queries.

MCQ traps

Trap	Correct answer
------	----------------

"In a B+ tree, data is stored in?"	Leaf nodes only
"B-tree of order m: max children / max keys?"	m children, m – 1 keys
"B-tree keeps balance by?"	Splitting on overflow, pushing the median up ; all leaves at the same level.
"Which structure links its leaves?"	B+ tree
"A B-tree of order m: min keys (non-root)?"	$\lceil m/2 \rceil - 1$
"Why B-trees for disk?"	Minimise disk accesses (short, high-fan-out tree)

12. Graphs

Concept

A **graph** $G = (V, E)$ is a set of **vertices** V and **edges** E connecting pairs of vertices.

Unlike a tree, a graph may have **cycles**, may be **disconnected**, and a vertex may have any number of neighbours. It is the most general non-linear structure.

Term	Meaning
Directed (digraph)	Edges have direction ($u \rightarrow v \neq v \rightarrow u$)
Undirected	Edges are bidirectional
Weighted	Each edge carries a cost/weight
Degree	Number of edges at a vertex (in-degree/out-degree for digraphs)
Path	A sequence of vertices joined by edges
Cycle	A path that returns to its start
Connected	A path exists between every pair (undirected)
Complete	Every pair of vertices is joined; $n(n-1)/2$ edges (undirected)
Adjacent	Two vertices joined by an edge

12.1 Representations — adjacency matrix vs adjacency list

Adjacency matrix: an $n \times n$ matrix A where $A[i][j] = 1$ (or the weight) if an edge $i \rightarrow j$ exists, else 0. Symmetric for undirected graphs.

Adjacency list: an array of n lists; list i holds the neighbours of vertex i .

Criterion	Adjacency matrix	Adjacency list
Space	$O(V^2)$ — wasteful for sparse graphs	$O(V + E)$ — compact for sparse graphs
Edge lookup "is $u \rightarrow v$ an edge?"	$O(1)$	$O(\text{degree})$
Iterate neighbours of u	$O(V)$	$O(\text{degree})$ — efficient

Add edge	$O(1)$	$O(1)$
Best for	Dense graphs, quick edge tests	Sparse graphs (the common case), traversals

12.2 BFS and DFS — worked traces

Breadth-First Search (BFS): explore **level by level** from the source, using a **QUEUE**.

Visit a vertex, enqueue all unvisited neighbours, repeat. Finds the **shortest path in an unweighted graph**. Time $O(V + E)$.

Depth-First Search (DFS): go **as deep as possible** before backtracking, using a **STACK** (or recursion). Time $O(V + E)$. Used for cycle detection, topological sort, connected components, path finding.

Worked. Graph (undirected), neighbours listed in ascending order:

```

1 — 2, 3
2 — 1, 4, 5
3 — 1, 6
4 — 2
5 — 2, 6
6 — 3, 5

```

BFS from 1 (queue in brackets):

| Dequeue | Enqueue unvisited neighbours | Visited order |

---|---|---

| 1 | 2, 3 → [2,3] | 1 |

| 2 | 4, 5 → [3,4,5] | 1 2 |

| 3 | 6 → [4,5,6] | 1 2 3 |

| 4 | — → [5,6] | 1 2 3 4 |

| 5 | (6 already queued) → [6] | 1 2 3 4 5 |

| 6 | — → [] | 1 2 3 4 5 6 |

BFS order = 1 2 3 4 5 6.

DFS from 1 (recursive / stack, take smallest unvisited first):

1 → 2 → 4 (dead end, back) → 5 → 6 → 3 (6's neighbour 3 unvisited) → back up, all done.

DFS order = 1 2 4 5 6 3.

12.3 Spanning trees and MST

A **spanning tree** of a connected graph with n vertices is a **subgraph that is a tree (no cycles) and includes all n vertices** — hence exactly **$n - 1$ edges**. A **minimum spanning tree (MST)** is the spanning tree of **least total edge weight**. Two greedy algorithms:

Prim's algorithm — grow **one tree** from a start vertex: repeatedly add the **cheapest edge that connects a vertex already in the tree to one outside it**. Good for dense graphs; $O(E \log V)$ with a heap.

Kruskal's algorithm — consider **all edges in ascending weight order**; add an edge if it **does not form a cycle** (checked with a **union-find / disjoint-set** structure). Stop at $n-1$ edges. Good for sparse graphs; $O(E \log E)$.

Worked MST. Weighted undirected graph:

```

Edges: A-B 4, A-C 4, B-C 2, B-D 5, C-D 8, C-E 10, D-E 2, D-F 6, E-F 3
Vertices: A B C D E F

```


Kruskal — sort edges: B-C 2, D-E 2, E-F 3, A-B 4, A-C 4, B-D 5, D-F 6, C-D 8, C-E 10.

| Edge | Weight | Cycle? | Action |

---|---|---|---

| B-C | 2 | no | **add** {B,C} |

| D-E | 2 | no | **add** {D,E} |

| E-F | 3 | no | **add** {D,E,F} |

| A-B | 4 | no | **add** {A,B,C} |

| A-C | 4 | **yes** (A,C already joined) | skip |

| B-D | 5 | no | **add** — now all vertices joined (5 edges = n-1) |

| stop | | | |

MST edges: B-C, D-E, E-F, A-B, B-D. Total weight = 2+2+3+4+5 = 16.

Prim from A gives the *same weight* (16): A-B(4), B-C(2), B-D(5), D-E(2), E-F(3). (MSTs may differ in edges if weights tie, but total weight is unique.)

12.4 Dijkstra's shortest path (WORKED TABLE)

Dijkstra finds the **shortest path from a single source to all vertices** in a graph with **non-negative weights**. Greedy: maintain a tentative distance for each vertex; repeatedly **finalise the unvisited vertex with the smallest tentative distance**, then **relax** its neighbours (update $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u,v))$).

Worked. Directed/undirected weighted graph, source = A:

A-B 4, A-C 1, C-B 2, C-D 5, B-D 1, B-E 5, D-E 3

Iteration	Finalised (dist)	A	B	C	D	E
init	—	0	∞	∞	∞	∞
pick A (0)	A	0	4	1	∞	∞
pick C (1)	A,C	0	3 (1+2)	1	6 (1+5)	∞
pick B (3)	A,C,B	0	3	1	4 (3+1)	8 (3+5)
pick D (4)	A,C,B,D	0	3	1	4	7 (4+3)
pick E (7)	all	0	3	1	4	7

Shortest distances from A: A=0, B=3, C=1, D=4, E=7.

Shortest path to E: A → C → B → D → E (1+2+1+3 = 7).

Key exam point: Dijkstra **fails with negative edge weights** — use **Bellman-Ford** there.

12.5 Topological sort

A **topological ordering** of a **DAG** (Directed Acyclic Graph) is a linear ordering of vertices such that for **every edge** $u \rightarrow v$, **u comes before v**. Used for **task scheduling**, **course prerequisites**, **build systems**, **instruction ordering**.

Two methods:

- **Kahn's (BFS-based):** repeatedly output a vertex with **in-degree 0**, remove it and decrement its neighbours' in-degrees.

- **DFS-based:** do DFS; push each vertex onto a stack **on finishing** (postorder); the reversed stack is the topological order.

Worked. Edges: A→C, B→C, C→D, D→E, B→D.

In-degrees: A=0, B=0, C=2, D=2, E=1. Output in-degree-0 vertices: **A** (C→1), then **B** (C→0, D→1), then **C** (D→0), then **D** (E→0), then **E**.

Topological order: A B C D E. (Multiple valid orders exist when several vertices have in-degree 0.)

Likely exam questions

- **[5]** Differentiate adjacency matrix and adjacency list; give space complexities.
- **[5]** Differentiate BFS and DFS; name their auxiliary structures.
- **[5]** Define spanning tree and minimum spanning tree.
- **[10]** Apply Kruskal's/Prim's algorithm to a given weighted graph and find the MST.
- **[10]** Apply Dijkstra's algorithm from a given source; show the distance table at each step.
- **[10]** Perform BFS and DFS on a given graph starting from a given vertex; show the order.
- **[15]** Explain graph representations, traversals (BFS, DFS with traces), and find the MST by both Prim and Kruskal, plus the shortest path by Dijkstra, for a given weighted graph.

MCQ traps

Trap	Correct answer
"BFS uses a? DFS uses a?"	Queue / Stack (or recursion).
"Adjacency matrix space?"	$O(V^2)$. Adjacency list: $O(V + E)$.
"MST of a graph with n vertices has?"	n – 1 edges.
"Kruskal avoids cycles using?"	Union-Find / disjoint set.
"Dijkstra fails when?"	There are negative edge weights (use Bellman-Ford).
"Topological sort applies to?"	A DAG (directed acyclic graph).
"BFS finds shortest path in?"	An unweighted graph.
"Prim grows? Kruskal grows?"	Prim: one tree . Kruskal: a forest merging into one.

13. Searching

13.1 Linear (sequential) search

Scan from the first element until the key is found or the list ends. Works on **any** list (sorted or not). **Best $O(1)$** (first element), **worst/average $O(n)$** . Simple but slow for large data.

13.2 Binary search (WORKED)

Requires a **sorted array**. Compare the key with the **middle** element; if equal, done; if the key is smaller, search the **left half**; if larger, the **right half**. Each step halves the search space → **$O(\log n)$** .

```
low = 0; high = n-1;
while (low <= high){
    mid = (low + high) / 2;
```

```

    if (A[mid] == key) return mid;
    else if (A[mid] < key) low = mid + 1;
    else high = mid - 1;
}
return -1;    /* not found */

```

Worked. Search 23 in [2, 5, 8, 12, 16, 23, 38, 45, 56, 72] (indices 0–9).

| Step | low | high | mid | A[mid] | Compare |

---|---|---|---|---|---

| 1 | 0 | 9 | 4 | 16 | 23 > 16 → low = 5 |

| 2 | 5 | 9 | 7 | 45 | 23 < 45 → high = 6 |

| 3 | 5 | 6 | 5 | 23 | **found at index 5** |

Three comparisons versus up to six for linear search — the gap widens hugely as n grows (1,000,000 elements ⇒ ≤ 20 comparisons).

	Linear search	Binary search
Data must be sorted?	No	Yes
Best case	O(1)	O(1)
Worst/avg case	O(n)	O(log n)
Data structure	Array or linked list	Array only (needs random access)

Likely exam questions

- **[5]** Compare linear and binary search.
- **[5]** Write the binary search algorithm and trace it for a given key.
- **[10]** Explain binary search (iterative and recursive); derive its O(log n) complexity.

MCQ traps

Trap	Correct answer
"Binary search requires the data to be?"	Sorted .
"Binary search time complexity?"	O(log n) .
"Binary search on a linked list?"	Inefficient — no O(1) random access.
"Linear search worst case?"	O(n) .

14. Sorting — every algorithm, worked, with the master table

Concept

Stability: a sort is **stable** if equal keys keep their original relative order — matters when sorting records by a secondary key. **In-place:** uses O(1) extra space. **Comparison sorts** are bounded below by $\Omega(n \log n)$; **non-comparison sorts** (counting, radix) beat that by exploiting key structure.

14.1 Bubble sort

Repeatedly step through the list, **swap adjacent out-of-order pairs**; the largest "bubbles" to the end each pass. With a swapped-flag, best case (already sorted) is $O(n)$.

Worked. 5 1 4 2 ascending.

- Pass 1: (5,1)→1 5 4 2; (5,4)→1 4 5 2; (5,2)→1 4 2 5. [5 fixed]
- Pass 2: (1,4)ok; (4,2)→1 2 4 5; [4 fixed]
- Pass 3: (1,2)ok. **Sorted: 1 2 4 5.**

14.2 Selection sort

Find the **minimum** of the unsorted part and swap it into the next position. Always $O(n^2)$; **minimises swaps** ($n-1$). Not stable.

Worked. 29 10 14 37 13:

min=10 → swap: 10 29 14 37 13; min(rest)=13 → 10 13 14 37 29; min=14 (in place); min=29 → 10 13 14 29 37. **Sorted.**

14.3 Insertion sort

Build the sorted portion one element at a time: take the next element and **insert it into its correct place** among the already-sorted elements by shifting. Best case $O(n)$ (nearly sorted), **stable**, great for small/almost-sorted data.

Worked. 12 11 13 5 6:

- 11 < 12 → 11 12 13 5 6
- 13 in place → 11 12 13 5 6
- 5 shifts to front → 5 11 12 13 6
- 6 inserts after 5 → 5 6 11 12 13. **Sorted.**

14.4 Merge sort

Divide and conquer: split the array in half, **recursively sort each half**, then **merge** the two sorted halves. Always $O(n \log n)$, **stable**, but needs $O(n)$ extra space for merging. The go-to for linked lists and external sorting.

Worked. 38 27 43 3 9 82 10:

- Split → [38 27 43 3] [9 82 10]
- Sort left → [3 27 38 43]; sort right → [9 10 82]
- Merge → compare fronts repeatedly → **3 9 10 27 38 43 82.**

14.5 Quick sort

Pick a **pivot**, **partition** so that smaller elements go left and larger go right, then recurse on each side. Average $O(n \log n)$ and usually the **fastest in practice** (good cache behaviour, in-place). **Worst case $O(n^2)$** when the pivot is always the smallest/largest (e.g. already-sorted data with first-element pivot) — mitigated by random or median-of-three pivots. Not stable.

Worked. 7 2 1 6 8 5 3 4, pivot = last (4), Lomuto partition:

Elements < 4: 2, 1, 3 → left; ≥ 4: 7, 6, 8, 5 → right; pivot 4 lands between:

2 1 3 | 4 | 7 6 8 5 → recurse each side → sorted 1 2 3 4 5 6 7 8.

14.6 Heap sort

Covered in §10.2: build a max-heap, repeatedly extract the max. **$O(n \log n)$ in all cases**, in-place, **not stable**.

3. **Comparison-sort lower bound is $\Omega(n \log n)$** ; counting/radix beat it because they don't compare keys.

4. **Insertion (and bubble with flag) are $O(n)$ on already-sorted data**; quick sort with a first-element pivot is **$O(n^2)$ on already-sorted data** — the classic trap pair.

Likely exam questions

- **[5]** Define stability and in-place sorting; classify the standard sorts.
- **[5]** Trace bubble/selection/insertion sort on a given list.
- **[10]** Explain merge sort with an example and derive its $O(n \log n)$ complexity.
- **[10]** Explain quick sort; show partitioning on a given array; discuss best and worst cases.
- **[15]** Compare all major sorting algorithms on time, space, stability and in-place property in a table, and trace any two of them on a given data set.

MCQ traps

Trap	Correct answer
"Which sort is $O(n^2)$ in the worst case but usually fastest?"	Quick sort .
"Worst case of quick sort?"	$O(n^2)$ (already-sorted with bad pivot).
"Which comparison sort is $O(n \log n)$ worst-case AND in-place?"	Heap sort .
"Which sorts are stable?"	Bubble, insertion, merge, counting, radix.
"Which sort does the fewest swaps?"	Selection ($n-1$).
"Non-comparison sorts?"	Counting, radix (also bucket).
"Best case of insertion sort?"	$O(n)$ (nearly sorted).
"Lower bound for comparison sorting?"	$\Omega(n \log n)$.

15. Hashing

Concept

Hashing maps a key directly to an array index via a **hash function** $h(\text{key})$, giving **average $O(1)$** insert, search and delete — beating the $O(\log n)$ of trees. A **hash table** is the array; a **collision** is when two keys hash to the same slot, and **collision resolution** is the heart of the topic.

Load factor $\alpha = n / m$ (n = keys stored, m = table size). It measures fullness: performance degrades as α rises. Rule of thumb — **keep α below ~ 0.7** for open addressing; rehash (grow and re-insert) when exceeded.

15.1 Hash functions

Method	$h(k)$	Note

Division	$k \bmod m$	Simplest; choose m prime (and not near a power of 2) to spread keys
Mid-square	Square k, take the middle digits	Depends on all digits of k
Folding	Split k into parts, add them, mod m	Good for long keys
Multiplication	$\lfloor m (kA \bmod 1) \rfloor$, $A \approx 0.618$	m need not be prime

15.2 Collision resolution

(a) Separate chaining — each slot holds a **linked list** of all keys hashing there.

Insert = prepend ($O(1)$); search = scan the chain ($O(1 + \alpha)$ average). Simple, α can exceed 1, deletion easy. Extra memory for pointers.

(b) Open addressing — all keys live **in the table itself**; on collision, **probe** for the next free slot by a fixed sequence. $\alpha \leq 1$ always. Three probe sequences:

Scheme	Probe i ($i = 0, 1, 2, \dots$)	Problem
Linear probing	$(h(k) + i) \bmod m$	Primary clustering — long runs of filled slots form and grow
Quadratic probing	$(h(k) + i^2) \bmod m$	Secondary clustering ; may not probe all slots
Double hashing	$(h_1(k) + i \cdot h_2(k)) \bmod m$	Best spread : needs a good second hash, e.g. $h_2(k) = R - (k \bmod R)$, R prime $< m$

15.3 Worked insertions

Table size $m = 10$, $h(k) = k \bmod 10$. Insert 23, 43, 13, 27, 33.

Linear probing $(h + i) \bmod 10$:

| Key | $h(k)$ | Probe sequence | Slot |

---|---|---|---

| 23 | 3 | slot 3 free | **3** |

| 43 | 3 | 3 taken → 4 free | **4** |

| 13 | 3 | 3,4 taken → 5 free | **5** |

| 27 | 7 | slot 7 free | **7** |

| 33 | 3 | 3,4,5 taken → 6 free | **6** |

Table: $[_, _, _, 23, 43, 13, 33, 27, _, _]$ — note the **cluster** 3–7.

Quadratic probing $(h + i^2) \bmod 10$ for the same keys:

| Key | h | Probes ($i^2=0,1,4,9,\dots$) | Slot |

---|---|---|---

| 23 | 3 | 3 | **3** |

| 43 | 3 | 3(taken), $3+1=4$ | **4** |

| 13 | 3 | 3,4 taken, $3+4=7$ | **7** |

| 27 | 7 | 7 taken, $7+1=8$ | **8** |

| 33 | 3 | 3,4 taken, $3+4=7$ taken, $3+9=12 \bmod 10=2$ | **2** |

Double hashing $h_2(k) = 7 - (k \bmod 7)$, probe $(h_1 + i \cdot h_2) \bmod 10$:

| Key | $h_1=k \bmod 10$ | $h_2=7-(k \bmod 7)$ | Probes | Slot |

---|---|---|---|---

23	3	7-2=5	3	**3**
43	3	7-1=6	3 taken → 3+6=9	**9**
13	3	7-6=1	3 taken → 4	**4**
27	7	7-6=1	7	**7**
33	3	7-5=2	3 taken → 5	**5**

Separate chaining — simply hang each key on the list at $k \bmod 10$: slot 3 → 23→43→13→33 (a chain of 4), slot 7 → 27.

15.4 Chaining vs open addressing

Criterion	Chaining	Open addressing
Storage	Table + linked lists	Table only
Load factor α	Can exceed 1	Must stay ≤ 1
Deletion	Easy	Hard (needs tombstones)
Clustering	None	Linear/quadratic clustering
Cache performance	Poorer (pointer chasing)	Better (contiguous)
Extra memory	Pointers	None

Likely exam questions

- **[5]** What is hashing? Define hash function, collision and load factor.
- **[5]** Explain linear probing, quadratic probing and double hashing.
- **[5]** Differentiate separate chaining and open addressing.
- **[10]** Insert the keys 12, 22, 32, 42, 52 into a table of size 10 using (a) linear probing and (b) chaining; show the final table.
- **[10]** Explain collision resolution techniques with worked examples and compare them.

MCQ traps

Trap	Correct answer
"Load factor is?"	$\alpha = n / m$ (keys / slots).
"Average time for hash search?"	$O(1)$.
"Linear probing suffers from?"	Primary clustering .
"Which resolution can have $\alpha > 1$?"	Separate chaining .
"Best hash-table size choice?"	A prime number (for division method).
"Double hashing probe?"	$(h_1(k) + i \cdot h_2(k)) \bmod m$.
"Deletion is awkward in?"	Open addressing (needs tombstones).

16. File structures / file organisation

Concept

A **file** is a named collection of **records**; each record is a set of **fields**; a field is a group of characters representing one data item. A **key field** uniquely identifies a record.

File organisation is how records are physically arranged on secondary storage and how they are accessed — the on-disk analogue of choosing a data structure.

Organisation	How records are stored	Access	Pros	Cons
Sequential	Records stored one after another , usually sorted on a key	Read in order; must scan from the start	Simple; efficient for batch processing and full-file reads; compact	No fast random access ; insert/delete needs rewriting/reorganising
Relative / Direct (random)	Record address computed directly from the key (often by hashing)	Direct access to any record in $\sim O(1)$	Very fast random access	Needs a key→address function; collisions; poor for sequential/range reads
Indexed	Data file + a separate index (key → record address), the index searched (often a B+ tree)	Random and ordered	Fast random access without hashing; supports range queries	Index storage and maintenance overhead
Index-sequential (ISAM)	Records sorted sequentially , plus an index of key ranges/blocks	Both sequential and indexed (random)	Best of both — ordered scans <i>and</i> keyed lookups	Index maintenance; overflow areas on insertion

16.1 Inverted lists (inverted files) — named in the Forensic syllabus

An **inverted list/index** maps **each attribute value** → **the list of record keys that have that value** — the inverse of "record → its field values". It is the structure behind **search engines and full-text search**: for the word "forensic", store the list of all documents/records containing it. Fast **multi-attribute and content queries** ("all records where dept = CS AND year = 2026") without scanning the whole file; cost is the storage and update of the indexes.

16.2 Multi-lists — named in the Forensic syllabus

A **multi-list** organisation threads records into **multiple linked lists**, one **per indexed attribute**. Each record carries a **link field for every attribute list** it belongs to, so it can be reached along any of those chains. Example: one chain links all "CS" records, another links all "2026" records; a record in CS-2026 sits on both chains. Contrast with inverted lists, where the pointers live in the **index** rather than **inside the records**.

	Inverted list	Multi-list
Where pointers live	In the index (a full list of keys per value)	Inside each record (one link per attribute)
Index size	Large (variable-length lists)	Small (index just points to each chain's head)
Query speed	Faster (list is precomputed)	Must traverse chains
Update cost	Higher (rebuild lists)	Lower (splice a node)

Likely exam questions

- **[5]** Define field, record and file. Differentiate sequential and direct file organisation.
- **[5]** What is an index-sequential file? State its advantages.
- **[5]** Differentiate inverted lists and multi-lists.
- **[10]** Explain the various file organisation techniques (sequential, direct/relative, indexed, index-sequential) with their access methods, advantages and disadvantages.

MCQ traps

Trap	Correct answer
"Best organisation for batch processing?"	Sequential .
"Direct/relative file access is achieved by?"	Hashing / key → address computation .
"ISAM supports?"	Both sequential and random access.
"Search engines use?"	Inverted lists/indexes .
"Pointers stored inside records characterise?"	Multi-list organisation.
"Smallest unit — field, record or file?"	Field .

17. Algorithm analysis and asymptotic notation

Concept

An **algorithm's** efficiency is measured by how its **running time / space grows with the input size n** — the machine-independent, constants-ignored view. We describe growth with **asymptotic notation**:

Notation	Bounds	Meaning	Says
Big-O — $O(f)$	Upper	Grows no faster than f	Worst case guarantee
Big-Omega — $\Omega(f)$	Lower	Grows at least as fast as f	Best case / lower bound
Big-Theta — $\Theta(f)$	Tight	Grows exactly like f (both bounds)	Exact order when best = worst

Formally, $T(n) = O(f(n))$ if there exist constants $c > 0$ and n_0 such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$. We **drop constants and lower-order terms**: $3n^2 + 5n + 7 = O(n^2)$.

The growth hierarchy (slowest-growing = best, left to right):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Class	Name	Typical source
$O(1)$	Constant	Array index, hash lookup, stack push
$O(\log n)$	Logarithmic	Binary search, balanced-tree operation

$O(n)$	Linear	Linear search, one loop over the data
$O(n \log n)$	Linearithmic	Merge/heap sort, efficient comparison sorts
$O(n^2)$	Quadratic	Bubble/selection/insertion sort, nested loops
$O(2^n)$	Exponential	Naive subset generation, brute-force TSP
$O(n!)$	Factorial	Permutation generation, brute-force TSP

Best vs average vs worst case: for the same algorithm these can differ — e.g. quick sort is $\Theta(n \log n)$ average but $O(n^2)$ worst; linear search is $O(1)$ best, $O(n)$ worst. **Big-O quoted alone almost always means the worst case.**

Design paradigms (named in the CS-Degree syllabus, worth a sentence each):

- **Divide and conquer** — split, solve recursively, combine (merge sort, quick sort, binary search).
- **Greedy** — make the locally optimal choice each step (Prim, Kruskal, Dijkstra, Huffman).
- **Dynamic programming** — solve overlapping subproblems once and store results (Floyd-Warshall, LCS, knapsack).
- **NP-completeness** — the class of problems for which no known polynomial algorithm exists and a solution is verifiable in polynomial time (SAT, TSP-decision, subset-sum).

Likely exam questions

- **[5]** Define big-O, big- Ω and big- Θ notation.
- **[5]** State the order of growth of common complexity classes with examples.
- **[5]** Give the best, average and worst-case complexity of quick sort and binary search.
- **[10]** Explain asymptotic notation with formal definitions and examples; analyse the time complexity of a given code fragment.

MCQ traps

Trap	Correct answer
"Big-O denotes the ____ bound?"	Upper (worst case).
"Which is faster: $O(n \log n)$ or $O(n^2)$?"	$O(n \log n)$.
" $3n^2 + 2n + 1$ is?"	$O(n^2)$.
"Binary search is?"	$O(\log n)$.
" Θ gives which bound?"	Tight (both upper and lower) .
"Merge sort belongs to which class?"	$O(n \log n)$.

18. Last-week revision sheet

Twenty-five facts most likely to be one-mark MCQs:

1. Stack = **LIFO**; queue = **FIFO**; deque = both ends; both stack & queue emulated by a deque.
2. **1D address**: $B + (i - LB) \cdot w$. **2D row-major** multiplies row index by **#columns (n)**; **column-major** by **#rows (m)**.
3. Postfix of $A+B \cdot C = A \ B \ C \ * \ +$; prefix = $+ \ A \ * \ B \ C$. Postfix/prefix need **no parentheses**.
4. **Recursion uses the call stack**; needs a **base case**; risk = **stack overflow**.
5. Circular-queue full (one-slot method): $(rear+1) \% n == front$.
6. Array = $O(1)$ access, $O(n)$ insert; linked list = $O(n)$ access, $O(1)$ insert. **Random access** → **array only**.
7. Tree with n nodes = **$n - 1$ edges**; max nodes at level $\ell = 2^\ell$; max nodes height $h = 2^{(h+1)} - 1$.
8. **Inorder of a BST = sorted**. Preorder root **first**, postorder root **last**.
9. Unique reconstruction needs **inorder + (pre or post)**; **pre + post fails** for general trees.
10. **AVL balance factor** $\in \{-1, 0, +1\}$; LL → right, RR → left, LR → left-then-right, RL → right-then-left.
11. **Heap = complete tree**; max-heap root = max; heap sort **$O(n \log n)$** , in-place, **not stable**; build-heap **$O(n)$** .
12. **B-tree order m** : $\leq m$ children, $\leq m-1$ keys, all leaves same level, split pushes **median up**.
13. **B+ tree**: data in **leaves only**, **leaves linked** → fast range queries.
14. **BFS = queue** (shortest path, unweighted); **DFS = stack/recursion**. Both **$O(V+E)$** .
15. Adjacency matrix **$O(V^2)$** ; adjacency list **$O(V+E)$** .
16. **MST has $n-1$ edges**; Prim grows one tree, Kruskal sorts edges + union-find.
17. **Dijkstra needs non-negative weights**; else **Bellman-Ford**.
18. **Topological sort** → **DAG** only.
19. **Binary search** → **sorted array, $O(\log n)$** .
20. **$O(n^2)$ sorts**: bubble, selection, insertion. **$O(n \log n)$** : merge, heap always; quick on average.
21. **Quick sort worst = $O(n^2)$** ; **merge & heap worst = $O(n \log n)$** .
22. **Stable**: bubble, insertion, merge, counting, radix. **Unstable**: selection, quick, heap, shell.
23. **Counting/radix** are **non-comparison** → beat $\Omega(n \log n)$.
24. **Load factor $\alpha = n/m$** ; linear probing → **primary clustering**; double hashing = best spread.
25. **Big-O = upper/worst**; **Ω = lower/best**; **Θ = tight**. Growth: $1 < \log n < n < n \log n < n^2 < 2^n < n!$.

Descriptive-paper strategy. Data structures reliably supply Section A short answers and at least one Section C 15-marker in the CS-Degree and Forensic papers. Prepare these six as guaranteed long answers, each with its worked example or diagram:

- (a) **Stacks — infix-to-postfix conversion + postfix evaluation, fully worked**;
- (b) **AVL tree — all four rotations + a build-from-sequence trace**;
- (c) **Graph algorithms — BFS/DFS traces + Prim/Kruskal + a Dijkstra table** on one graph;
- (d) **Sorting — the master comparison table + a trace of merge and quick sort**;
- (e) **Hashing — collision resolution with a worked linear-probing / chaining insertion**;
- (f) **B-tree / B+ tree — construction with splitting + the B-tree vs B+ tree table**.

For every numerical (address calc, postfix, Dijkstra, hashing, sort trace) **show every**

intermediate step in a table — worked numericals are unambiguous to mark and score the highest per minute of writing.

● Degree — programming C/C++

Source: `_Shared/CORE_09_ProgrammingC_CPP.md`

CORE 09 — Programming in C and C++ / OOP

Shared-core file 9 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points), **10/15-mark answer** (worked explanation + code snippet with expected output).

0. Why this matters — where C/C++ appears in your three papers

Elective	Where	How heavy
CS Degree	P-I §1 "Basics and Programming — Programming in C/C++: Data types, declarations and expressions, functions, pointers, arrays."	Heavy
Computer Forensic	TP-II §4 — a full unit : "Elements of C ... Structured data types — arrays, struct, union, string, pointers. OOP Concepts : class, object, inheritance, polymorphism, overloading. C++ : tokens, datatypes, operators, control statements, functions, parameter passing, class and objects, constructors and destructor, overloading, inheritance, templates, exception handling."	Very heavy
CS Diploma	P-II §5 "Programming in C" — history, algorithms, flowcharts, operators & hierarchy, I/O (<code>printf/scanf/getchar/putchar/getch/putch</code>), control statements, loops, and (per the source PDF) arrays, functions, pointers, structures, file handling.	Heavy

All three electives test C; two of them test C++/OOP. The Diploma is strong on **C fundamentals and I/O**; the Forensic paper is strong on **the full OOP feature set**; the Degree paper on **pointers, arrays and functions**. Study the union once. **Every code snippet in an answer should state its expected output** — examiners reward a correctly predicted output more than prose.

Topic checklist — C

- [] Features & history; program structure; compilation stages
- [] Tokens: keywords, identifiers, constants, strings, operators, separators
- [] Data types and their **sizes**; type modifiers; type conversion (implicit/explicit)
- [] **Operators + full precedence/associativity table**
- [] I/O: `printf`, `scanf`, `getchar`, `putchar`, `getch`, `putch` + **format specifiers**
- [] Control: `if`, `if-else`, nested `if`, `switch`, `goto`
- [] Loops: `for`, `while`, `do-while`, `break`, `continue`
- [] Arrays & strings; **string functions**
- [] Functions: **call by value vs reference**, recursion, **storage classes table**

- [] **Pointers**: arithmetic, pointers & arrays, pointer-to-pointer, function pointers
- [] **malloc / calloc / realloc / free** differences
- [] **Structures vs unions**
- [] File handling
- [] Preprocessor and macros

Topic checklist — C++ / OOP

- [] Four pillars: encapsulation, abstraction, inheritance, polymorphism
- [] Classes & objects; access specifiers
- [] **Constructors (default/parameterised/copy) & destructors**
- [] Function overloading & **operator overloading**
- [] **Inheritance types + diamond problem + virtual base classes**
- [] **Virtual functions, vtable, runtime polymorphism**
- [] Compile-time vs run-time polymorphism
- [] Abstract classes / pure virtual functions
- [] Templates (function & class)
- [] Exception handling
- [] Friend functions; static members; `this` pointer; inline functions

PART A — THE C LANGUAGE

1. Features, history and program structure

Concept

C was developed by **Dennis Ritchie at Bell Labs in 1972** to write the **UNIX** operating system. It is a **general-purpose, procedural, structured, middle-level** language — "middle" because it combines **high-level** readability with **low-level** access to memory (pointers, bit operations), making it ideal for system software, compilers, embedded systems and OS kernels. Standardised as **ANSI C (C89/C90)**, then **C99, C11, C17**.

Key features (list these for a 5-marker): portable, fast/efficient, structured (functions & blocks), rich set of operators and built-in functions, **pointers** (direct memory access), **dynamic memory allocation**, recursion, extensible (user libraries), small core with a large standard library, case-sensitive.

"Middle-level" is the single most examined descriptor — say it.

1.1 Structure of a C program

```
#include <stdio.h>           /* 1. Preprocessor directive (header) */
#define PI 3.14159           /* 2. Symbolic constant / macro          */

int square(int x);           /* 3. Function prototype (declaration) */

int main(void) {             /* 4. main() — execution starts here */
    int n = 5;               /* Local declarations              */
}
```

```

    printf("%d\n", square(n)); /* Statements */
    return 0; /* Return status to the OS */
}

int square(int x){ /* 5. Function definition */
    return x * x;
}

```

Expected output: 25

Compilation stages (worth a diagram): **source (.c)** → **Preprocessor** (expands `#include`, `#define`) → **Compiler** (produces assembly/object `.o`) → **Assembler** → **Linker** (joins object files + libraries) → **executable (.exe / a.out)** → **Loader** (loads into memory to run).

Likely exam questions

- [5] State any six features of C. Why is it called a middle-level language?
- [5] Explain the structure of a C program with an example.
- [5] Explain the stages of compilation of a C program.

MCQ traps

Trap	Correct answer
"Who developed C and when?"	Dennis Ritchie, 1972, Bell Labs.
"C was written to develop?"	The UNIX OS.
"C is a ____ level language?"	Middle level.
"Execution of a C program begins at?"	main() .
" <code>#include</code> is processed by the?"	Preprocessor .

2. Tokens — keywords, identifiers, constants

Concept

A **token** is the smallest individual unit of a C program. There are **six** kinds: **keywords, identifiers, constants, string literals, operators, and special symbols (separators)**.

Keywords — 32 reserved words in ANSI C (`int`, `float`, `if`, `else`, `while`, `for`, `return`, `struct`, `union`, `const`, `static`, `void`, `sizeof`, ...). They have fixed meaning and **cannot be used as identifiers**.

Identifiers — names given to variables, functions, arrays, etc. **Rules:**

1. Begin with a **letter or underscore** (`_`), never a digit.
2. Followed by letters, digits or underscores only.
3. **No spaces, no special symbols, no keywords.**
4. **Case-sensitive** (`Sum` ≠ `sum`).

Constants (literals): integer (`75`, `0x4B` hex, `075` octal), floating (`3.14`, `2.5e3`), character (`'A'` — stored as its ASCII value), string (`"NPSC"` — an array of chars ending in `'\0'`). A constant can be fixed with `const` or `#define`.

Escape sequences: `\n` newline, `\t` tab, `\0` null, `\\` backslash, `\'` quote, `\"`, `\r`, `\b`, `\a` bell.

MCQ traps

Trap	Correct answer
"Number of keywords in ANSI C?"	32.
"Valid identifier: <code>2sum</code> , <code>_sum</code> , <code>su m</code> , <code>for</code> ?"	<code>_sum</code> (others break a rule).
"A character constant is stored as its?"	ASCII value.
"A string ends with?"	The null character <code>\0</code> .
"Is C case-sensitive?"	Yes.

3. Data types, sizes and type conversion

Concept

C data types classify variables by the kind and size of value they hold.

Category	Types
Basic (primary)	<code>char</code> , <code>int</code> , <code>float</code> , <code>double</code> , <code>void</code>
Derived	array, pointer, function
User-defined	<code>struct</code> , <code>union</code> , <code>enum</code> , <code>typedef</code>

Type modifiers: `signed`, `unsigned`, `short`, `long` change range/size.

3.1 Typical sizes (32/64-bit systems) — MCQ material

Type	Size (bytes)	Format specifier	Typical range
<code>char</code>	1	<code>%c</code>	−128 to 127 (signed)
<code>unsigned char</code>	1	<code>%C</code>	0 to 255
<code>short int</code>	2	<code>%hd</code>	−32,768 to 32,767
<code>int</code>	4 (usually)	<code>%d</code>	−2.1×10 ⁹ to 2.1×10 ⁹
<code>unsigned int</code>	4	<code>%u</code>	0 to 4.29×10 ⁹
<code>long int</code>	4 or 8	<code>%ld</code>	platform-dependent

<code>long long int</code>	8	<code>%lld</code>	large
<code>float</code>	4	<code>%f</code>	~6–7 significant digits
<code>double</code>	8	<code>%lf</code>	~15–16 significant digits
<code>long double</code>	10/12/16	<code>%Lf</code>	extended

Use `sizeof(type)` to get the exact size on a given machine — never assume.

3.2 Type conversion

Implicit (type promotion / coercion) — the compiler auto-converts in a mixed expression, promoting toward the "wider" type: `char` → `int` → `unsigned` → `long` → `float` → `double` → `long double`. Example: `int / int` truncates, but `int / float` promotes the int to float.

Explicit (type casting) — the programmer forces a conversion: `(float)5 / 2 = 2.5`, whereas `5 / 2 = 2` (integer division). **Integer division truncates toward zero** — the classic trap.

```
#include <stdio.h>
int main(void){
    int a = 5, b = 2;
    printf("%d\n", a / b);           /* integer division */
    printf("%.2f\n", (float)a / b); /* cast → real division */
    return 0;
}
```

Expected output:

```
2
2.50
```

MCQ traps

Trap	Correct answer
"Size of <code>int</code> (typical)?"	4 bytes (2 on old 16-bit compilers).
"Size of <code>double</code> ?"	8 bytes .
" <code>5 / 2</code> gives?"	2 (integer division).
"Specifier for <code>double</code> in scanf?"	<code>%lf</code> .
"Operator to find size at compile time?"	<code>sizeof</code> .
"Widest basic type?"	<code>long double</code> .

4. Operators — with the precedence/associativity table

Concept

C has a rich operator set:

Category	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (<code>%</code> = modulo, integers only)
Relational	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>
Logical	<code>&&</code> <code> </code> <code>!</code> (short-circuit)
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> ...
Increment/decrement	<code>++</code> <code>--</code> (pre and post)
Conditional (ternary)	<code>cond ? a : b</code>
Others	<code>sizeof</code> , <code>&</code> (address), <code>*</code> (dereference), <code>.</code> , <code>-></code> , <code>,</code> (comma), <code>[]</code> , <code>()</code>

4.1 Precedence and associativity table (high → low)

Level	Operators	Associativity
1 (highest)	<code>()</code> <code>[]</code> <code>.</code> <code>-></code> (postfix <code>++</code> <code>--</code>)	Left → right
2	unary <code>!</code> <code>~</code> <code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>*</code> (deref) <code>&</code> (addr) <code>sizeof</code> (cast)	Right → left
3	<code>*</code> <code>/</code> <code>%</code>	Left → right
4	<code>+</code> <code>-</code>	Left → right
5	<code><<</code> <code>>></code>	Left → right
6	<code><</code> <code><=</code> <code>></code> <code>>=</code>	Left → right
7	<code>==</code> <code>!=</code>	Left → right
8	<code>&</code> (bitwise AND)	Left → right
9	<code>^</code> (bitwise XOR)	Left → right
10	<code> </code> (bitwise OR)	Left → right
11	<code>&&</code>	Left → right
12	<code> </code>	Left → right
13	<code>? :</code> (ternary)	Right → left
14	<code>=</code> <code>+=</code> <code>-=</code> ... (assignment)	Right → left
15 (lowest)	<code>,</code> (comma)	Left → right

The three facts examiners test: (1) unary, ternary and assignment are right-associative,

everything else in the middle is left-associative; (2) *** / % beat + -**;

(3) **arithmetic beats relational beats logical beats assignment**.

Worked evaluation. `int x = 5 + 3 * 2;` → * first → `5 + 6 = 11` (not 16).

`int y = 10 > 5 && 2 < 1;` → relational first → `1 && 0 = 0`.

4.2 Pre- vs post-increment (a guaranteed MCQ)

```
#include <stdio.h>
int main(void) {
    int a = 5, b;
    b = a++;      /* post: assign THEN increment → b=5, a=6 */
    printf("a=%d b=%d\n", a, b);
    int c = 5, d;
    d = ++c;      /* pre: increment THEN assign → d=6, c=6 */
    printf("c=%d d=%d\n", c, d);
    return 0;
}
```

Expected output:

```
a=6 b=5
c=6 d=6
```

Rule: post-increment uses the value first, then increments; pre-increment increments first, then uses.

MCQ traps

Trap	Correct answer
" <code>5 + 3 * 2 = ?</code> "	11 (* before +).
" <code>%</code> works on?"	Integers only (not float).
"Which operators are right-associative?"	Unary, ternary <code>?:</code>, assignment.
" <code>a = b = 5</code> evaluates?"	Right to left (assignment associativity).
" <code>i++</code> returns?"	The old value, then increments.
"Ternary operator?"	<code>?:</code> — the only ternary operator in C.

5. Input / output and format specifiers

Concept

C I/O is via the **standard library** `<stdio.h>` (formatted) and `<conio.h>` (console, Turbo C).

Function	Purpose	Header
----------	---------	--------

<code>printf</code>	Formatted output to screen	<code>stdio.h</code>
<code>scanf</code>	Formatted input from keyboard (needs <code>&</code> for variables)	<code>stdio.h</code>
<code>getchar</code>	Read a single char (echoed , needs Enter)	<code>stdio.h</code>
<code>putchar</code>	Write a single char	<code>stdio.h</code>
<code>getch</code>	Read a char without echo, no Enter needed	<code>conio.h</code>
<code>putch</code>	Write a char to console	<code>conio.h</code>
<code>gets</code> / <code>puts</code>	Read/write a string (whole line)	<code>stdio.h</code>

`getch()` VS `getchar()` — the classic distinction: `getch()` reads a key **immediately, without echoing it** to the screen (used for "Press any key..."); `getchar()` **echoes** and waits for Enter.

5.1 Format specifiers

Specifier	Data type
<code>%d</code> / <code>%i</code>	int (decimal)
<code>%u</code>	unsigned int
<code>%f</code>	float / double (printf)
<code>%lf</code>	double (scanf)
<code>%c</code>	char
<code>%s</code>	string
<code>%x</code> / <code>%X</code>	hexadecimal
<code>%o</code>	octal
<code>%e</code>	scientific notation
<code>%p</code>	pointer address
<code>%%</code>	a literal <code>%</code>

Width/precision: `%5d` (min width 5), `%-5d` (left-justified), `%.2f` (2 decimals), `%6.2f` (width 6, 2 decimals).

```
#include <stdio.h>
int main(void){
    int n; float pi = 3.14159f;
    printf("Enter a number: ");
    scanf("%d", &n);          /* & is essential */
    printf("n = %d, pi = %.2f\n", n, pi);
    return 0;
```

```
}
```

If input is `7`, **expected output**: `n = 7, pi = 3.14`

The `&` in `scanf` is compulsory for basic variables (it passes the address so `scanf` can write back) — omitting it is the most common runtime crash and a favourite MCQ.

MCQ traps

Trap	Correct answer
"scanf needs which operator before a variable?"	<code>&</code> (address-of).
" <code>getch()</code> differs from <code>getchar()</code> how?"	No echo, no Enter needed.
"Specifier to print a string?"	<code>%s</code> .
" <code>%.2f</code> prints?"	Float with 2 decimal places .
" <code>getch/putch</code> are declared in?"	<code>conio.h</code> .
"Print a literal percent sign?"	<code>%%</code> .

6. Control statements and loops

Concept — decision making

Statement	Form
<code>if</code>	<code>if(cond){ ... }</code>
<code>if-else</code>	<code>if(cond){...} else {...}</code>
Nested if / else-if ladder	<code>if(c1){} else if(c2){} else {}</code>
<code>switch</code>	multiway branch on an integer/char constant
<code>goto</code>	unconditional jump to a label (discouraged)

`switch` rules (examined): the expression must be **integer or char** (not float, not string); each `case` label must be a **constant**; `break` is needed or execution **falls through** to the next case; `default` is optional and handles no-match.

```
#include <stdio.h>
int main(void){
    int day = 3;
    switch(day){
        case 1: printf("Mon\n"); break;
        case 2: printf("Tue\n"); break;
        case 3: printf("Wed\n"); break;
        default: printf("Other\n");
```

```

    }
    return 0;
}

```

Expected output: Wed. (Remove the `breaks` and it would print `Wed Other` — the fall-through trap.)

6.1 Loops

Loop	When the condition is tested	Runs at least once?
<code>for(init; cond; update)</code>	Before each iteration (entry-controlled)	No
<code>while(cond)</code>	Before each iteration (entry-controlled)	No
<code>do { } while(cond);</code>	After each iteration (exit-controlled)	Yes — at least once

do-while is the only exit-controlled loop — it always executes the body once even if the condition is false initially. That is the single most examined loop fact.

- `break` — exits the **innermost** loop/`switch` immediately.
- `continue` — skips the rest of the current iteration and goes to the next.

```

#include <stdio.h>
int main(void) {
    for(int i = 1; i <= 5; i++){
        if(i == 3) continue; /* skip 3 */
        if(i == 5) break;    /* stop before 5 */
        printf("%d ", i);
    }
    return 0;
}

```

Expected output: 1 2 4

Likely exam questions

- [5] Differentiate `while` and `do-while` loops with examples.
- [5] Explain the `switch` statement and the role of `break` and `default`.
- [5] Differentiate `break` and `continue`.
- [10] Explain the decision-making and looping statements in C with syntax and examples.

MCQ traps

Trap	Correct answer
"Which loop is exit-controlled?"	<code>do-while</code> (runs at least once).
" <code>switch</code> works on which types?"	<code>int</code> / <code>char</code> (not float or string).

"Missing <code>break</code> in switch causes?"	Fall-through .
" <code>continue</code> does?"	Skips to the next iteration .
" <code>break</code> exits?"	The innermost loop or switch.
"Entry-controlled loops?"	for and while .

7. Arrays and strings

Concept

An **array** in C is a fixed-size, contiguous collection of same-type elements, indexed from **0**. `int a[5];` reserves 5 ints (`a[0]...a[4]`). 2-D: `int m[3][4];`. C does **no bounds checking** — accessing `a[5]` is undefined behaviour (a common exam warning).

A **string** in C is simply a **character array terminated by the null character `\0`**. `char s[6] = "HELLO";` uses 6 bytes (5 letters + `\0`). There is **no separate string type**.

7.1 String functions (<string.h>) — memorise

Function	Action
<code>strlen(s)</code>	Length excluding <code>\0</code>
<code>strcpy(d, s)</code>	Copy s into d
<code>strcat(d, s)</code>	Append s to d
<code>strcmp(a, b)</code>	Compare: 0 if equal , <0 if a<b if a>b (lexicographic)
<code>strncpy/strncat/strncmp</code>	Length-limited versions
<code>strrev(s)</code>	Reverse (Turbo C)
<code>strupr/strlwr</code>	Upper/lower case (Turbo C)
<code>strstr(s, t)</code>	Find substring t in s

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char a[20] = "Data";
    char b[] = "Structure";
    strcat(a, b);
    printf("%s (len %d)\n", a, strlen(a));
    printf("cmp=%d\n", strcmp("abc", "abd"));
    return 0;
}
```

Expected output:

```
DataStructure (len 13)
cmp=-1
```

(`strcmp` returns the sign of the difference of the first differing characters: 'c'-'d' = -1.)

MCQ traps

Trap	Correct answer
"Array indexing starts at?"	0.
"A C string ends with?"	<code>\0</code> (null).
" <code>strlen("HELLO")</code> = ?"	5 (excludes <code>\0</code> , but storage needs 6).
" <code>strcmp</code> returns 0 when?"	Strings are equal .
"Does C check array bounds?"	No.
"String functions are in?"	<code><string.h></code> .

8. Functions — call by value/reference, recursion, storage classes

Concept

A **function** is a self-contained block that performs a task; it promotes **modularity**, **reusability and readability**. Three parts: **declaration (prototype)**, **definition**, and **call**. A function may **return** one value (via `return`) and receive several **parameters**.

8.1 Call by value vs call by reference

	Call by value	Call by reference
What is passed	A copy of the argument's value	The address of the argument (a pointer)
Effect on original	Cannot change the caller's variable	Can modify the caller's variable
Default in C?	Yes — C is call-by-value	Simulated using pointers (<code>&</code> and <code>*</code>)
Overhead	Copies data	Passes an address (cheap for large data)

C is strictly call-by-value. "Call by reference" in C is *achieved by passing pointers* — this nuance is heavily examined. (True reference parameters `&` exist only in **C++**.)

```
#include <stdio.h>
void swapVal(int a, int b){ int t=a; a=b; b=t; }          /* no effect */
void swapRef(int *a, int *b){ int t=*a; *a=*b; *b=t; }    /* swaps */
```

```
int main(void){
    int x=10, y=20;
    swapVal(x, y); printf("value: x=%d y=%d\n", x, y);
    swapRef(&x, &y); printf("ref   : x=%d y=%d\n", x, y);
    return 0;
}
```

Expected output:

```
value: x=10 y=20
ref   : x=20 y=10
```

8.2 Recursion

A function that **calls itself** with a smaller input until a **base case** stops it (see also DataStructures §4.4). Every call adds a stack frame.

```
#include <stdio.h>
int fib(int n){
    if(n < 2) return n;          /* base case */
    return fib(n-1) + fib(n-2);  /* recursive case */
}
int main(void){ printf("%d\n", fib(6)); return 0; }
```

Expected output: 8 (sequence 0 1 1 2 3 5 8 → fib(6)=8).

8.3 Storage classes — the table

A **storage class** defines a variable's **scope** (visibility), **lifetime** (how long it lives), **default initial value**, and **storage location**.

Storage class	Keyword	Scope	Lifetime	Default value	Stored in
Automatic	<code>auto</code>	Local (block)	Until block ends	Garbage	Stack
Register	<code>register</code>	Local (block)	Until block ends	Garbage	CPU register (if free)
Static	<code>static</code>	Local: block; retains value between calls . Global: file	Entire program	Zero	Data segment
External	<code>extern</code>	Global (across files)	Entire program	Zero	Data segment

`static` local variable — the exam favourite: it is **initialised once** and **retains its value between function calls**, unlike `auto`.

```
#include <stdio.h>
void counter(void){
```



```

static int c = 0;    /* initialised once, persists */
c++;
printf("%d ", c);
}
int main(void){ counter(); counter(); counter(); return 0; }

```

Expected output: 1 2 3 (with `auto int c` it would print 1 1 1).

Likely exam questions

- **[5]** Differentiate call by value and call by reference with an example.
- **[5]** Explain the four storage classes in C with scope, lifetime and default value.
- **[5]** What is recursion? Write a recursive function for factorial.
- **[10]** Explain functions in C — declaration, definition, call, parameter passing methods — with the swap example, and describe storage classes in a table.

MCQ traps

Trap	Correct answer
"C passes arguments by?"	Value (reference simulated via pointers).
" <code>static</code> local variable default value?"	0 , and it retains value between calls .
" <code>auto</code> variable default value?"	Garbage .
"Fastest-access storage class?"	<code>register</code> .
" <code>extern</code> is used for?"	Global variables across files.
"Recursion needs a?"	Base case .

9. Pointers

Concept

A **pointer** is a variable that **stores the memory address** of another variable. `int *p = &a;` makes `p` point to `a`; `*p` (**dereference**) accesses the value at that address; `&a` (**address-of**) gives `a`'s address. Pointers give C its power: **dynamic memory, call-by-reference, efficient arrays/strings, data structures (linked lists, trees)**.

```

int a = 10;
int *p = &a;    /* p holds the address of a */
printf("%d", *p); /* prints 10 - the value AT that address */

```

9.1 Pointer arithmetic

Pointer arithmetic is **scaled by the size of the pointed-to type**. If `p` points to an `int`

(4 bytes) at address 2000, then `p + 1` is **2004**, not 2001 — it moves to the next *element*.
Valid: `++`, `--`, `+ n`, `- n`, and subtracting two pointers (gives the element count between them). **Invalid**: adding two pointers, multiplying/dividing pointers.

9.2 Pointers and arrays

An array name **is** a constant pointer to its first element: `a[i]` is exactly `*(a + i)`.
So `a`, `&a[0]`, and `p = a` are equivalent starting points. This equivalence is examined constantly.

```
#include <stdio.h>
int main(void){
    int a[] = {10, 20, 30};
    int *p = a;                /* points to a[0] */
    printf("%d %d\n", *(p+1), p[2]); /* 20 30 */
    return 0;
}
```

Expected output: 20 30

9.3 Pointer to pointer

A **double pointer** stores the address of a pointer: `int **pp = &p;`. `*pp` gives `p`, `**pp` gives the final value. Used for 2-D dynamic arrays and modifying a pointer inside a function.

9.4 Function pointers

A **function pointer** holds the address of a function, enabling **callbacks** and jump tables:

```
int add(int a, int b){ return a + b; }
int (*fp)(int, int) = add;      /* fp points to add */
printf("%d", fp(3, 4));        /* prints 7 */
```

9.5 Dynamic memory — malloc / calloc / realloc / free

Declared in `<stdlib.h>`. Memory is taken from the **heap** and **must be freed manually** — C has no garbage collector. Forgetting `free` causes a **memory leak**.

Function	Purpose	Initialises memory?	Arguments
<code>malloc(size)</code>	Allocate <code>size</code> bytes	No (contains garbage)	one: total bytes
<code>calloc(n, size)</code>	Allocate an array of <code>n</code> elements	Yes — all zeros	two: count, element size
<code>realloc(ptr, size)</code>	Resize a previously allocated block	Preserves old data	pointer, new size
<code>free(ptr)</code>	Release the block back to the heap	—	pointer

The examined differences: `malloc` leaves **garbage**, `calloc` **zero-initialises**;
`malloc` takes **one** argument, `calloc` takes **two**; both return a `void*` (cast if needed)
and **NULL on failure** (always check).

```

#include <stdio.h>
#include <stdlib.h>
int main(void){
    int *arr = (int*)calloc(3, sizeof(int));    /* {0,0,0} */
    arr[0] = 5; arr[1] = 10;
    arr = (int*)realloc(arr, 5 * sizeof(int)); /* grow to 5 */
    arr[4] = 99;
    printf("%d %d %d\n", arr[0], arr[1], arr[4]);
    free(arr);                                /* release */
    return 0;
}

```

Expected output: 5 10 99

Likely exam questions

- **[5]** What is a pointer? Explain `&` and `*` with an example.
- **[5]** Explain pointer arithmetic. Why is `p+1` not `address+1`?
- **[5]** Differentiate `malloc` and `calloc`.
- **[10]** Explain dynamic memory allocation in C with `malloc`, `calloc`, `realloc` and `free`, giving a program.
- **[10]** Explain the relationship between pointers and arrays with examples; explain pointer-to-pointer and function pointers.

MCQ traps

Trap	Correct answer
" <code>&</code> operator gives?"	The address of a variable.
" <code>*p</code> is the?"	Value at the address (dereference).
" <code>p + 1</code> for an int pointer moves by?"	sizeof(int) bytes (e.g. 4).
" <code>a[i]</code> is equivalent to?"	* (a + i) .
" <code>calloc</code> vs <code>malloc</code> ?"	<code>calloc</code> zero-initialises and takes two args.
"malloc returns on failure?"	NULL .
"Not freeing memory causes?"	A memory leak .
"Functions for dynamic memory are in?"	<stdlib.h> .

10. Structures vs unions

Concept

A **structure** (`struct`) groups **different-type** variables (members) under one name — a user-defined record. A **union** looks identical syntactically but **all members share the same memory location**.

Criterion	struct	union
Memory	Sum of all members' sizes (+ padding)	Size of the largest member only
Members active at once	All simultaneously	Only one at a time
Use	Store a full record (all fields together)	Save memory when only one field is used at a time
Changing one member	Others unaffected	Overwrites the shared memory

```
#include <stdio.h>
struct S { int i; char c; double d; }; /* 4 + 1 + 8 (+pad) */
union U { int i; char c; double d; }; /* size of double = 8 */
int main(void) {
    printf("struct=%lu union=%lu\n", sizeof(struct S), sizeof(union U));
    return 0;
}
```

Expected output (typical): `struct=16 union=8` (struct padded to 16; union = largest member, 8). Access members with `.` on a variable and `->` on a pointer to it.

MCQ traps

Trap	Correct answer
"Union size equals?"	Its largest member.
"Struct size equals?"	Sum of members (plus padding).
"In a union, how many members hold valid data at once?"	One .
"Access member via a pointer?"	-> operator.
"Which saves memory?"	Union .

11. File handling

Concept

File handling lets a program store data **permanently** on disk. Uses a **FILE *** pointer (from `<stdio.h>`). Workflow: **fopen** → **read/write** → **fclose**.

Modes: `"r"` read, `"w"` write (truncate/create), `"a"` append, `"r+"/"w+"/"a+"` read-write; add `"b"` for binary (`"rb"`, `"wb"`).

Function	Purpose
<code>fopen(name, mode)</code>	Open; returns <code>NULL</code> on failure
<code>fclose(fp)</code>	Close and flush
<code>fprintf / fscanf</code>	Formatted write/read
<code>fputc / fgetc</code>	Char write/read
<code>fputs / fgets</code>	String write/read
<code>fwrite / fread</code>	Block (binary) write/read
<code>fseek / ftell / rewind</code>	Random access / position
<code>feof(fp)</code>	End-of-file test

```
#include <stdio.h>
int main(void){
    FILE *fp = fopen("out.txt", "w");
    if(fp == NULL){ printf("Cannot open\n"); return 1; }
    fprintf(fp, "NPSC 2026\n");
    fclose(fp);
    return 0;
}
```

Writes `NPSC 2026` to `out.txt`. Always check `fopen` for `NULL`.

MCQ traps

Trap	Correct answer
"File pointer type?"	<code>FILE *</code>
"Mode <code>"w"</code> on an existing file?"	Truncates it (overwrites).
"fopen returns on failure?"	<code>NULL</code> .
"Append mode?"	<code>"a"</code> .
"End-of-file check?"	<code>feof()</code> .

12. Preprocessor and macros

Concept

The **preprocessor** runs **before compilation**, acting on lines starting with `#`. It does

text substitution — it does not know C syntax.

Directive	Purpose
<code>#include</code>	Insert a header file (<...> system, "... " user)
<code>#define</code>	Define a macro / symbolic constant
<code>#undef</code>	Remove a macro
<code>#ifdef</code> / <code>#ifndef</code> / <code>#endif</code>	Conditional compilation (include guards)
<code>#if</code> / <code>#elif</code> / <code>#else</code>	Conditional compilation on constant expressions
<code>#pragma</code>	Compiler-specific instruction

Object-like macro: `#define PI 3.14`. **Function-like macro:** `#define SQ(x) ((x)*(x))`.

The macro-parenthesis trap: `#define SQ(x) x*x` then `SQ(2+3)` expands to `2+3*2+3 = 11`, not 25. Always parenthesise: `#define SQ(x) ((x)*(x))` → `((2+3)*(2+3)) = 25`. Macros are **not functions** — no type checking, no evaluation, just text replacement (and no semicolon).

```
#include <stdio.h>
#define SQ(x) ((x)*(x))
int main(void){ printf("%d\n", SQ(2+3)); return 0; }
```

Expected output: 25

MCQ traps

Trap	Correct answer
"Preprocessor runs?"	Before compilation.
" <code>#define</code> does?"	Text substitution.
" <code>SQ(x) x*x</code> , <code>SQ(2+3) = ?</code> "	11 (missing parentheses trap).
"Include guards use?"	<code>#ifndef</code> / <code>#define</code> / <code>#endif</code> .
" <code><stdio.h></code> vs <code>"myfile.h"</code> ?"	System header vs user header.

PART B — C++ AND OBJECT-ORIENTED PROGRAMMING

13. The four pillars of OOP

Concept

C++ (Bjarne Stroustrup, Bell Labs, **1979**, originally "C with Classes") extends C with **object-oriented** features. OOP models software as interacting **objects** — bundles of **data (attributes)** and **functions (methods)** — rather than procedures acting on data. Its four foundational principles:

Pillar	Meaning	Achieved by
Encapsulation	Binding data and the functions that operate on it into one unit (class), and hiding internal state	Classes + private members
Abstraction	Exposing only the essential features , hiding implementation detail	Public interface; abstract classes; header/implementation split
Inheritance	A class (derived) reuses and extends another (base)	<code>class D : public B</code>
Polymorphism	" Many forms " — one interface, many behaviours	Overloading (compile-time), virtual functions (run-time)

Data hiding (via **private**) is the mechanism; **encapsulation** is the concept — a common MCQ distinction. **Procedural (C) vs OOP (C++)**: procedural is function-centred with global, exposed data; OOP is object-centred with data hidden inside objects and accessed through methods.

MCQ traps

Trap	Correct answer
"Who developed C++ / when?"	Bjarne Stroustrup, 1979 (Bell Labs).
"Wrapping data + functions into one unit?"	Encapsulation .
"Showing only essential features?"	Abstraction .
"Many forms'?"	Polymorphism .
"Data hiding is enforced by?"	The private access specifier.
"C++ was originally called?"	" C with Classes ".

14. Classes, objects and access specifiers

Concept

A **class** is a **user-defined type** — a blueprint combining data members and member functions. An **object** is an **instance** of a class (memory is allocated only when an object is created). The class describes; the object exists.

Access specifiers control member visibility:

Specifier	Accessible from
private (default for class)	Only within the class and its friends
protected	Within the class and derived classes
public	Anywhere the object is visible

(A **struct** in C++ is a class whose members default to **public**.)

```
#include <iostream>
using namespace std;
class Rectangle {
    int w, h; // private by default
public:
    void set(int a, int b){ w=a; h=b; }
    int area(){ return w*h; }
};
int main(){
    Rectangle r; // object
    r.set(4, 5);
    cout << r.area() << endl;
    return 0;
}
```

Expected output: 20

MCQ traps

Trap	Correct answer
"Default access in a class ?"	private .
"Default access in a struct (C++)?"	public .
"An object is a(n)?"	Instance of a class.
" protected members are visible in?"	The class and derived classes .
"Memory is allocated when?"	An object is created (not at class definition).

15. Constructors and destructors

Concept

A **constructor** is a special member function with the **same name as the class**, **no return type** (not even `void`), **called automatically** when an object is created, used to initialise data. A **destructor** (`~ClassName`, no args, no return) is called automatically when the object is destroyed, to release resources.

Type of constructor	Signature	Purpose
Default	<code>Box()</code>	No parameters; sets defaults
Parameterised	<code>Box(int l, int w)</code>	Initialise with supplied values
Copy	<code>Box(const Box &b)</code>	Create a new object as a copy of an existing one

Copy constructor is invoked when: an object is initialised from another (`Box b2 = b1;`), an object is **passed by value**, or **returned by value**. If you don't write one, the compiler supplies a **shallow copy** — dangerous when the class holds pointers (both copies free the same memory → the reason to write a **deep-copy** constructor).

```
#include <iostream>
using namespace std;
class Box {
    int v;
public:
    Box(){ v = 0; cout << "default\n"; }           // default
    Box(int x){ v = x; cout << "param " << v << "\n"; } // parameterised
    Box(const Box &b){ v = b.v; cout << "copy " << v << "\n"; } // copy
    ~Box(){ cout << "destroy " << v << "\n"; }      // destructor
};
int main(){
    Box a;           // default
    Box b(7);        // parameterised
    Box c = b;       // copy
    return 0;
}
```

Expected output:

```
default
param 7
copy 7
destroy 7
destroy 7
destroy 0
```

(Destructors run in **reverse order** of construction: c, b, a.)

MCQ traps

Trap	Correct answer
"A constructor returns?"	Nothing — no return type.
"Constructor name equals?"	The class name .
"Copy constructor parameter?"	A reference (<code>const Box&</code>) — passing by value would recurse infinitely.
"Destructors run in what order?"	Reverse of construction.
"Can a constructor be overloaded?"	Yes (default, parameterised, copy).
"Default copy constructor does a?"	Shallow copy .

16. Function and operator overloading (compile-time polymorphism)

Concept

Overloading = the same name with **different parameter lists**, resolved by the compiler at **compile time** (**static / early binding**) — this is **compile-time polymorphism**.

Function overloading: several functions share a name but differ in **number or type of parameters** (return type alone is **not** enough to overload).

```
int add(int a, int b){ return a+b; }
double add(double a, double b){ return a+b; }
int add(int a, int b, int c){ return a+b+c; }
```

Operator overloading: give an existing operator (**+**, **-**, **==**, **<<**, ...) new meaning for **user-defined types**, so objects can be used with natural syntax.

```
#include <iostream>
using namespace std;
class Complex {
    int re, im;
public:
    Complex(int r=0, int i=0): re(r), im(i) {}
    Complex operator+(const Complex &c){          // overload +
        return Complex(re + c.re, im + c.im);
    }
    void show(){ cout << re << "+" << im << "i\n"; }
};

int main(){
    Complex a(2,3), b(4,1);
    Complex c = a + b;    // calls operator+
    c.show();
    return 0;
}
```

```
}
```

Expected output: `6+4i`

Operators that **CANNOT** be overloaded (a guaranteed MCQ): `.` (dot), `::` (scope resolution), `?:` (ternary), `.*` (pointer-to-member), and `sizeof`.

MCQ traps

Trap	Correct answer
"Overloading is resolved at?"	Compile time (static/early binding).
"Can functions differ by return type alone?"	No — parameters must differ.
"Which operators can't be overloaded?"	<code>., ::, ?, .*, sizeof</code> .
"Operator overloading works on?"	User-defined types (classes).
"Overloading is which polymorphism?"	Compile-time .

17. Inheritance, the diamond problem and virtual base classes

Concept

Inheritance lets a **derived class** reuse and extend a **base class**, modelling an "is-a" relationship (a Car *is a* Vehicle). Syntax: `class Derived : access Base { ... };`

17.1 Types of inheritance

Type	Structure
Single	One base → one derived ($A \rightarrow B$)
Multiple	One derived, several bases ($A, B \rightarrow C$)
Multilevel	Chain ($A \rightarrow B \rightarrow C$)
Hierarchical	One base → several derived ($A \rightarrow B, A \rightarrow C$)
Hybrid	A combination of the above (often multiple + hierarchical → the diamond)

17.2 Mode of inheritance — how base access changes

Base member	<code>public</code> inheritance	<code>protected</code> inheritance	<code>private</code> inheritance
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>

protected	protected	protected	private
private	not inherited (inaccessible)	inaccessible	inaccessible

Private members of the base are never directly accessible in the derived class (only via the base's public/protected functions) — an examined point.

17.3 The diamond problem and virtual base classes

The **diamond problem** arises in **multiple/hybrid inheritance**: if class **D** inherits from both **B** and **C**, and both **B** and **C** inherit from a common base **A**, then **D ends up with two copies of A's members** — an ambiguity (which `A::x?`).



Solution: virtual base classes. Declare A as a **virtual base** in B and C:

`class B : virtual public A { ... };` and `class C : virtual public A { ... };`. Now D inherits **only one shared copy** of A, removing the ambiguity.

```

class A { public: int x; };
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };    // one shared A – no ambiguity
  
```

Likely exam questions

- **[5]** Define inheritance. Explain its types with diagrams.
- **[5]** Explain the visibility modes (public/protected/private inheritance).
- **[10]** What is the diamond problem? Explain how virtual base classes solve it, with code.
- **[10]** Explain the types of inheritance with syntax and an example each.

MCQ traps

Trap	Correct answer
"Diamond problem occurs in?"	Multiple / hybrid inheritance.
"Diamond problem is solved by?"	Virtual base classes .
"Are base's private members inherited?"	Not accessible in the derived class.
"Inheritance models which relationship?"	"is-a" .

"A → B → C is which type?"	Multilevel·
"One base, many derived?"	Hierarchical·

18. Virtual functions, vtables and run-time polymorphism

Concept

Run-time polymorphism = deciding **which function to call at run time**, based on the actual object type — **dynamic / late binding**. It is achieved with **virtual functions** through a **base-class pointer or reference**.

The mechanism: declare a member **virtual** in the base and **override** it in derived classes. When you call it through a base pointer that actually points to a derived object, **the derived version runs**. Without **virtual**, the call is bound at compile time to the **base** version (the pointer's static type) — this contrast is the single most examined C++ topic.

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void draw(){ cout << "Shape\n"; }    // virtual
};
class Circle : public Shape {
public:
    void draw(){ cout << "Circle\n"; }           // override
};
int main(){
    Shape *p = new Circle();
    p->draw();           // run-time: calls Circle::draw
    return 0;
}
```

Expected output: **Circle** (with **virtual** removed, it would print **Shape**).

How it works — the vtable. For every class with virtual functions the compiler builds a **vtable** (virtual function table): an array of pointers to that class's virtual functions. Each object of such a class holds a hidden **vp**tr pointing to its class's vtable. A virtual call is resolved by **following the object's vp**tr into the **vtable** at run time — so the *actual* object's function is invoked. This is the implementation behind late binding.

18.1 Compile-time vs run-time polymorphism

	Compile-time (static)	Run-time (dynamic)
--	-----------------------	--------------------

Also called	Early binding	Late binding
Achieved by	Function/operator overloading , templates	Virtual functions (via base pointer/ref)
Bound when	At compile time	At run time
Speed	Faster	Slight overhead (vtable lookup)
Flexibility	Less	More

18.2 Abstract classes and pure virtual functions

A **pure virtual function** has **no body** and is declared `= 0`:

`virtual void draw() = 0;`. A class containing at least one pure virtual function is an **abstract class** — it **cannot be instantiated**; it exists only to be inherited, forcing derived classes to **provide** the implementation (an enforced interface). A derived class that fails to override all pure virtuals is itself abstract.

```
class Shape {                                // abstract
public:
    virtual double area() = 0;               // pure virtual
};
class Square : public Shape {
    double s;
public:
    Square(double x): s(x) {}
    double area(){ return s*s; }           // must override
};
```

Likely exam questions

- **[5]** What is a virtual function? Why is it needed?
- **[5]** Differentiate compile-time and run-time polymorphism.
- **[5]** What is an abstract class / pure virtual function?
- **[10]** Explain run-time polymorphism with virtual functions and a program; describe how the vtable and vptr implement dynamic binding.
- **[15]** Explain polymorphism in C++ in full: overloading, virtual functions, abstract classes, and the vtable mechanism, with programs and expected output.

MCQ traps

Trap	Correct answer
"Virtual functions enable?"	Run-time (dynamic) polymorphism.

"Late binding is implemented via?"	The vtable / vpitr .
"A pure virtual function is declared with?"	= 0 .
"A class with a pure virtual function is?"	Abstract — cannot be instantiated.
"Without virtual , base-pointer call binds to?"	The base version (compile time).
"Can a constructor be virtual?"	No . (A destructor should be virtual in a base class.)

19. Templates, exceptions, friends, static, **this**, inline

19.1 Templates — generic programming

A **template** lets you write a function or class **once for any data type**, the type supplied at compile time. This is **generic programming** and a form of **compile-time polymorphism**.

```
#include <iostream>
using namespace std;
template <class T>
T maxOf(T a, T b){ return (a > b) ? a : b; }
int main(){
    cout << maxOf(3, 7) << " " << maxOf(2.5, 1.5) << " " << maxOf('a','z') << endl;
    return 0;
}
```

Expected output: 7 2.5 z

Class template: `template <class T> class Stack { T data[100]; ... };.` The C++ **STL** (vector, list, map) is built entirely on templates.

19.2 Exception handling

C++ separates **error handling** from normal logic using **try** / **throw** / **catch**: code that may fail goes in **try**; on an error it **throws** an exception object; a matching **catch** block handles it. This avoids error-code clutter and lets errors propagate up the call stack.

```
#include <iostream>
using namespace std;
int main(){
    int a = 10, b = 0;
    try {
        if(b == 0) throw "Divide by zero!";
        cout << a / b;
    } catch(const char *msg){
```

```

        cout << "Error: " << msg << endl;
    }
    return 0;
}

```

Expected output: Error: Divide by zero!

A `catch(...)` block catches **any** exception. Order catch blocks specific → general.

19.3 Friend functions

A **friend function** is a **non-member** function granted access to a class's **private and protected** members. Declared inside the class with the `friend` keyword but defined outside; it is **not** called on an object (no `this`). Used when a function needs the internals of **two** classes, or for operator overloading like `<<`. **Friendship is not inherited and not mutual** — examined points.

19.4 Static members

- **Static data member: one copy shared by all objects** of the class (a class-wide variable, e.g. an object counter); must be **defined outside** the class.
- **Static member function:** can be called **without an object** (`ClassName::func()`); can access **only static members** (it has **no `this` pointer**).

```

#include <iostream>
using namespace std;
class Counter {
    static int count;           // shared
public:
    Counter(){ count++; }
    static int get(){ return count; }
};
int Counter::count = 0;        // definition
int main(){
    Counter a, b, c;
    cout << Counter::get() << endl;    // called without an object
    return 0;
}

```

Expected output: 3

19.5 The `this` pointer

Every **non-static** member function receives a hidden pointer `this` holding the **address of the object** on which it was called. Used to disambiguate a member from a parameter of the same name (`this->x = x;`) and to **return the object itself** for method chaining


```
(return *this;).
```

19.6 Inline functions

An **inline** function requests the compiler to **replace the call with the function body**, eliminating call overhead — best for **small, frequently-called** functions. **inline** is only a **request**; the compiler may ignore it (e.g. for large or recursive functions). Member functions **defined inside the class body are implicitly inline**.

Likely exam questions

- [5] What is a template? Write a template function for finding the maximum.
- [5] Explain exception handling with **try**, **throw**, **catch**.
- [5] What is a friend function? State its characteristics.
- [5] Explain static data members and static member functions.
- [5] What is the **this** pointer? Give one use.
- [10] Explain templates and exception handling in C++ with programs and expected output.

MCQ traps

Trap	Correct answer
"Templates support?"	Generic (type-independent) programming .
"Exception is raised by?"	throw ; handled by catch .
" catch(...) catches?"	Any exception.
"A friend function is a?"	Non-member with access to private members; no this .
"Static data member has how many copies?"	One , shared by all objects.
"A static member function has a this pointer?"	No .
" this pointer holds?"	The address of the current object .
" inline is a?"	Request to the compiler (may be ignored).

20. Last-week revision sheet

Thirty facts most likely to be one-mark MCQs:

1. **C = Dennis Ritchie, 1972, for UNIX; middle-level, procedural. C++ = Stroustrup, 1979.**
2. Execution starts at **main()**; **#include/#define** handled by the **preprocessor**.
3. **32 keywords** in ANSI C; identifiers can't start with a digit; C is **case-sensitive**.

4. **int = 4 B, double = 8 B, float = 4 B, char = 1 B**; `sizeof` gives the exact size.
5. **5 / 2 = 2** (integer division); cast for real division.
6. Precedence: **unary > * / % > + - > relational > && > || > ?: > =**; unary, ternary, assignment are **right-associative**.
7. **Post-increment uses-then-increments; pre-increment increments-then-uses.**
8. **scanf needs &**; `getch()` = no echo, no Enter; `getchar()` echoes.
9. **do-while is the only exit-controlled loop** — runs at least once.
10. **switch on int/char only**; missing `break` = fall-through.
11. **break exits the loop; continue skips to the next iteration.**
12. Arrays index from **0**; **no bounds checking**; a string ends in `\0`.
13. `strlen` excludes `\0`; `strcmp` returns **0 when equal**.
14. **C is call-by-value**; "call by reference" = passing **pointers**.
15. Storage classes: `auto` (garbage, stack), `register` (CPU), **static (0, persists between calls)**, `extern` (global).
16. **p+1** for an int pointer moves `sizeof(int)` bytes; `a[i] == *(a+i)`.
17. **malloc = garbage, 1 arg; calloc = zeroed, 2 args; realloc resizes; free releases** — all in `<stdlib.h>`; return **NULL** on failure.
18. **struct = sum of members; union = largest member; union shares one memory.**
19. File pointer is `FILE *`; mode `"w"` truncates; check **NULL** after `fopen`.
20. **Macro trap**: `#define SQ(x) x*x`, `SQ(2+3)=11`; parenthesise for 25.
21. **Four pillars**: encapsulation, abstraction, inheritance, polymorphism; **data hiding via private**.
22. Default access: `class` → **private**, `struct` → **public**.
23. **Constructor**: class name, no return type, auto-called; **destructor ~**, reverse order.
24. **Copy constructor takes const T&**; default copy = **shallow**.
25. **Overloading = compile-time (early) polymorphism**; can't overload by return type alone.
26. **Cannot overload**: `::?:.* sizeof`.
27. **Diamond problem** → **virtual base classes**. Inheritance = **"is-a"**.
28. **Virtual functions** → **run-time (late) binding via the vtable/vptr**.
29. **Pure virtual = 0** → **abstract class** → **cannot be instantiated**. Destructor should be virtual; constructor cannot be.
30. **Static member = one shared copy; static function has no this; this = address of the current object**.

Descriptive-paper strategy. C/C++ reliably supplies Section A short answers and at least one Section C 15-marker in all three papers. Prepare these six as guaranteed long answers, each with a short program and its **stated expected output**:

- (a) **Pointers + dynamic memory** (`malloc/calloc/realloc/free`) with the swap example;
- (b) **Storage classes** table + the `static` counter program;
- (c) **Structures vs unions** with the `sizeof` program;

(d) **Constructors & destructors** (all three constructors + destructor order);

(e) **Inheritance + the diamond problem + virtual base classes**;

(f) **Polymorphism** — overloading vs virtual functions + the vtable explanation.

Always **write the program, then predict its output line by line** — a correctly traced output scores more reliably than description, and is quick to mark.